

Universitatea „Sapientia” din Cluj-Napoca

FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU-MUREȘ

SPECIALIZAREA CALCULATOARE

Aplicarea inteligenței artificiale pentru
optimizarea jocurilor pe calculator

PROIECT DE DIPLOMĂ

Coordonator științific:

Ș.l.dr.ing. Kovács Lehel István

Absolvent:

György Ákos

2026

Aplicarea inteligenței artificiale pentru optimizarea jocurilor pe calculator

Extras

Această lucrare prezintă dezvoltarea unui sistem software integrat pentru jocul Snake, cu scopul comparării strategiilor de control clasice și bazate pe inteligență artificială. Soluția este implementată într-o arhitectură monorepo și include trei componente principale: un frontend React + TypeScript pentru interfața jocului și experiența utilizatorului, un backend Node.js + Express + SQLite pentru autentificare și stocarea rezultatelor, respectiv un serviciu AI Python FastAPI pentru decizii în timp real prin WebSocket.

Stratul AI conține atât strategii euristice (A*, BFS, Hamilton, lookahead, minimax, max safety), cât și strategii învățate (DQN, PPO, Neuroevolution/NEAT), toate integrate printr-o interfață unificată de tip „stare de joc in, direcție out”. Pentru evaluare obiectivă, proiectul utilizează un benchmark reproductibil, cu seed-uri controlate, care măsoară scorul mediu și median, numărul de pași, cauzele de terminare și procentul de atingere a primului obiectiv (hrană). Rezultatele arată că, în configurația documentată, strategiile euristice oferă performanțe mai stabile decât modelele învățate, subliniind importanța alegerii funcției de recompensă, a reprezentării stării și a parametrilor de antrenare.

Sapientia Erdélyi Magyar Tudományegyetem

MAROSVÁSÁRHELYI KAR

SZÁMÍTÁSTECHNIKA SZAK

Mesterséges intelligencia alkalmazása
számítógépes játékok optimalizálásához

DIPLOMADOLGOZAT

Témavezető:

Dr. Kovács Lehel István
egyetemi adjunktus

Végzős hallgató:

György Ákos

2026

Kivonat

A dolgozat egy Snake játékra épülő, mesterséges intelligencia stratégiákat vizsgáló webes rendszert mutat be. A megvalósítás három együttműködő komponensre épül: React + TypeScript frontend a játékmenethez és felhasználói felülethez, Node.js + Express + SQLite backend az autentikációhoz és pontszámkezeléshez, valamint Python FastAPI alapú AI szolgáltatás a valós idejű stratégiai döntésekhez. A rendszer különlegessége, hogy egységes interfészen kezeli a heurisztikus és tanuló algoritmusokat.

A heurisztikus stratégiák között szerepel az A*, BFS, Hamilton-jellegű bejárás, lookahead, minimax és biztonságvezérelt megközelítés; a tanuló ágban DQN, PPO és NEAT alapú modell is elérhető. A benchmark modul reprodukálható seed-ekkel, rögzített protokoll szerint hasonlítja össze az algoritmusokat. A mért mutatók között megtalálható az átlag- és mediánpontszám, lépésszám, halálokok megoszlása, valamint az első étel elérésének aránya. Az eredmények alapján a vizsgált konfigurációban a heurisztikák stabilabb teljesítményt adnak, míg a tanuló modellek eredménye erősen függ a jutalmazástól, állapotreprezentációtól és a tanítási hiperparaméterektől. A dolgozat hozzájárulása egy olyan gyakorlati és kutatható keretrendszer, amelyben az AI-vezérelt játékstratégiák közvetlenül összevethetők.

Abstract

This thesis presents an integrated software system built around the Snake game to evaluate and compare multiple AI control strategies under reproducible conditions. The implementation follows a monorepo architecture and consists of three major components: a React + TypeScript frontend for gameplay and user interaction, a Node.js + Express + SQLite backend for authentication and score persistence, and a Python FastAPI AI service that provides real-time movement decisions over WebSocket.

The AI layer includes both heuristic strategies (A*, BFS, Hamilton-style traversal, lookahead, minimax, and safety-oriented methods) and learning-based approaches (DQN, PPO, and Neuroevolution/NEAT), all exposed through a unified strategy interface. To ensure objective comparison, the project introduces a benchmark pipeline with controlled seeds and fixed protocol settings. Reported metrics include mean and median score, mean step count, death reason distribution, and first-food reach rate. In the documented configuration, heuristic methods outperform trained agents in stability and average score, while trained models show strong sensitivity to reward shaping, observation design, and training budget. The final outcome is both a playable web application and an experimentation framework suitable for AI strategy analysis.

Tartalomjegyzék

- [1. Bevezető](#)
- [2. Elméleti megalapozás és szakirodalmi tanulmány](#)
 - [2.1 Szakirodalmi tanulmány](#)
 - [2.2 Elméleti alapok](#)
 - [2.3 Ismert hasonló alkalmazások](#)
 - [2.4 Felhasznált technológiák](#)
 - [2.4.1 Frontend technológiák](#)
 - [2.4.2 Backend technológiák](#)
 - [2.4.3 AI service technológiák](#)
- [3. A rendszer specifikációi és architektúrája](#)
 - [3.1 Monorepo, fájlstruktúra](#)
 - [3.1.1 Monorepo áttekintés](#)
 - [3.1.2 Frontend fájlstruktúra](#)
 - [3.1.3 Backend fájlstruktúra](#)
 - [3.1.4 AI service fájlstruktúra](#)
 - [3.1.5 Benchmark fájlstruktúra](#)
 - [3.2 Frontend](#)
 - [3.2.1 Állapot és adatfolyam](#)
 - [3.2.2 Adatszerkezetek és stratégia interfész](#)
 - [3.2.3 Játékállapot](#)
 - [3.2.4 Perzisztencia és UI](#)
 - [3.2.5 Téma és auth kontextus](#)
 - [3.2.6 Statisztika oldal \(frontend nézet\)](#)
 - [3.3 Backend](#)
 - [3.3.1 Adatbázis és séma](#)
 - [3.3.2 JWT middleware](#)
 - [3.3.3 Végpontok és üzleti szabályok](#)
 - [3.4 AI service](#)
 - [3.4.1 Közös interfész és állapot](#)
 - [3.4.2 Heurisztikus stratégiák \(A*, BFS, FollowTail, Hamilton, Lookahead, Minimax\)](#)
 - [3.4.3 Tanult stratégiák \(DQN, PPO, NEAT\)](#)
 - [3.4.4 REST és WebSocket viselkedés](#)
 - [3.4.5 Stratégiák táblázatban](#)
 - [3.4.6 Tanuló környezet és algoritmusok \(SnakeEnv, DQN, PPO, NEAT\)](#)
 - [3.5 Benchmark](#)
 - [3.5.1 Benchmark szerepe az architektúrában](#)
 - [3.5.2 Adatkapcsolat a rendszer többi részével](#)
- [4. Tervezés folyamata](#)
 - [4.1 Első tervezési fázis – alkalmazásmag és integrációs alapok](#)
 - [4.1.1 Játék újraindítása játék vége után](#)
 - [4.1.2 Frontend és Python közös állapot- és döntési szerződés](#)
 - [4.1.3 Regisztráció és REST: „Failed to fetch”](#)
 - [4.1.4 Heurisztikus stratégiák bővítése](#)
 - [4.1.5 Három szolgáltatás párhuzamos indítása \(npm run dev:all\)](#)
 - [4.2 Második tervezési fázis – tanuló algoritmusok fejlesztése](#)
 - [4.2.1 DQN: megfigyelés-egyeztetés és tanítási érzékenység](#)
 - [4.2.2 PPO: környezet, reward finomhangolás és legjobb modell mentése](#)

- [4.2.3 NEAT implementálása és tanítása](#)
 - [4.3 Harmadik tervezési fázis – benchmark, statisztika és következtetések](#)
- [5. Üzembe helyezés és kísérleti eredmények](#)
 - [5.1 Üzembe helyezési lépések](#)
 - [5.2 Kísérleti eredmények, mérések](#)
 - [5.2.1 Cél és szerep](#)
 - [5.2.2 Futtató: `run\_strategy\_benchmark.py`](#)
 - [5.2.3 Kimenet: JSON és összefoglaló](#)
 - [5.2.4 Vizualizáció: `generate\_benchmark\_plots.py`](#)
 - [5.3 Eredmények részletes értelmezése](#)
 - [5.3.1 Mit jelentenek a metrikák?](#)
 - [5.3.2 Heurisztikák vs. tanult stratégiák \(a weboldal narratívája\)](#)
 - [5.3.3 Halálprofilok és szakmai értelmezés](#)
 - [5.3.4 Hatékonyság és robusztusság \(összehasonlító keret\)](#)
 - [5.3.5 Heurisztikák \(500 futás / stratégia, dokumentált kampány\)](#)
 - [5.3.6 Tanuló stratégiák \(200 futás / stratégia\)](#)
 - [5.3.7 Vizualizáció: generált grafikonok \(PNG\)](#)
 - [5.3.8 Záró következtetések \(kampány és szakmai útmutató\)](#)
- [6. A rendszer felhasználása és funkcionalitások](#)
 - [6.1 Áttekintés: egyoldalas navigáció és screen állapot](#)
 - [6.2 Navigáció és fejléc \(Header\)](#)
 - [6.3 Főmenü \(MainMenu\)](#)
 - [6.4 Bejelentkezés, regisztráció és auth perzisztencia](#)
 - [6.5 Témaválasztás \(világos / sötét\)](#)
 - [6.6 Beállítások \(Settings, PlayerSettings, AISettings\)](#)
 - [6.7 Játék képernyő – játékos mód](#)
 - [6.8 Játék képernyő – MI mód](#)
 - [6.9 Játék vége és pontszám mentés](#)
 - [6.10 Eredmények oldal \(Results\)](#)
 - [6.11 Statisztika oldal \(Statistics\) – összefoglaló](#)
 - [6.12 Profil oldal \(Profile\)](#)
- [7. Következtetések](#)
 - [7.1 Megvalósítások](#)
 - [7.2 Hasonló rendszerekkel való összehasonlítás](#)
 - [7.3 Továbbfejlesztési lehetőségek](#)
 - [7.3.1 Rendszer, telepítés és üzemeltetés](#)
 - [7.3.2 Benchmark-adatok, szinkron és reprodukálhatóság](#)
 - [7.3.3 Tanulási környezet, jutalomstruktúra és hiperparaméterek](#)
 - [7.3.4 Frontend, auth és felhasználói perzisztencia](#)
 - [7.3.5 Minőségbiztosítás, tesztelés és folyamatok](#)
- [8. Irodalomjegyzék](#)
- [9. Függelék](#)

Ábrák jegyzéke

1. `score_mean_bar.png` – átlagpontok összehasonlítása ([5.3.7](#))
2. `score_distribution_boxplot.png` – ponteloszlás (boxplot) ([5.3.7](#))
3. `death_distribution_stacked.png` – halálok megoszlása ([5.3.7](#))
4. `score_vs_steps_scatter.png` – pont és lépésszám kapcsolata ([5.3.7](#))
5. `images/headbar.png` – fejléc (Header): cím, auth és téma kapcsoló ([6.2](#))

6. `images/mainpage.png` – főmenü: módválasztó és navigáció ([6.3](#))
7. `images/registration.png` – regisztrációs űrlap ([6.4](#))
8. `images/lightmode.png` – világos téma megjelenítés ([6.5](#))
9. `images/settings.png` – beállítások képernyő ([6.6](#))
10. `images/game.png` – játék képernyő (HUD + pálya + vezérlők) ([6.7](#))
11. `images/userresultsexample.png` – eredmények oldal példa ([6.10](#))
12. `images/profilpage.png` – profil oldal ([6.12](#))

Táblázatok jegyzéke

1. Monorepo felépítés táblázat ([3.1.1](#))
2. Backend API végpontok táblázat ([3.3.3](#))
3. Stratégiák azonosító-táblázata ([3.4.5](#))
4. `state_to_observation` (12 dimenzió) összefoglaló táblázat ([3.4.6.1](#))
5. Generált benchmark plot fájlok és értelmezésük ([5.2.4](#))
6. Heurisztikák benchmark eredménytáblázata ([5.3.5](#))
7. Tanuló stratégiák benchmark eredménytáblázata ([5.3.6](#))

1. Bevezető

A dolgozat témája a mesterséges intelligencia módszereinek gyakorlati alkalmazása egy klasszikus, jól modellezhető játékproblémán, a **Snake** környezetben. A feladatterület pontosan behatárolt: rácsalapú, diszkrét idejű játékmenetben kell mozgásdöntéseket hozni úgy, hogy a kígyó minél több ételt gyűjtsön, miközben elkerüli a fal- és önműködést, valamint az éhezés miatti leállást. A téma azért alkalmas szakdolgozati szintű vizsgálatra, mert egyszerre adható hozzá intuitív felhasználói felület és szigorúan mérhető, reprodukálható kísérleti protokoll.

A dolgozat tulajdonképpen tervezési és kutatási célja egy olyan **integrált rendszer** megvalósítása, amelyben ugyanazon játékszabályok mellett közvetlenül összehasonlíthatók a különböző döntési paradigmák: klasszikus heurisztikák, megerősítéses tanuláson alapuló modellek és neuroevolúciós megközelítések. Ennek megfelelően a munka nem csak egy játék implementálására irányul, hanem egy komplett, többkomponensű szoftverarchitektúra létrehozására is: webes kliens a játékmenethez, backend az autentikációhoz és eredménytároláshoz, valamint külön AI szolgáltatás valós idejű stratégiai döntésekhez és benchmark-adatok kiszolgálásához.

A célkitűzések egyértelműen három szintre bonthatók. **(1) Funkcionális cél:** a felhasználó által ténylegesen használható webalkalmazás biztosítása (játékos és MI mód, beállítások, eredmények, profil, téma-váltás). **(2) Módszertani cél:** reprodukálható benchmark-folyamat kialakítása kontrollált seedekkel, azonos pályamérettel és közös mérőszámokkal, hogy az összehasonlítás ne szubjektív benyomásokra, hanem adatokra épüljön. **(3) Elemzési cél:** a kapott eredmények szakmai értelmezése több metrika mentén (átlag, medián, lépésszám, halálozoprofil, első étel elérésének aránya), nem csak egyetlen mutató alapján.

A témaválasztás személyes és szakmai indoka kettős. Egyrészt a Snake játék logikailag egyszerű, ezért a figyelem a döntéshozatal minőségére és a modellek viselkedésére terelhető, nem vész el komplex szabályrendszerben. Másrészt a projekt jól összeköti az egyetemi képzés több fontos területét: algoritmuselméletet (útkeresés, döntési stratégiák), mesterséges intelligenciát (RL, neuroevolution), webfejlesztést (frontend-backend integráció), valamint szoftvertervezési és üzemeltetési gyakorlatot (API-k, adatperzisztencia, moduláris rendszerfelépítés). Ez a kombináció a dolgozatot egyszerre teszi mérnöki és kutatási jellegűvé.

A dolgozat központi kérdése így fogalmazható meg: **milyen teljesítményt és viselkedésmintázatot mutatnak különböző MI- és heurisztikus stratégiák azonos Snake-környezetben, azonos mérési feltételek mellett?** A kérdéshez kapcsolódóan további alcél, hogy a rendszer képes legyen a stratégiák közötti váltásra, az eredmények tartós mentésére, valamint olyan vizualizációra, amely a fejlesztő és a felhasználó számára is értelmezhetően mutatja be a különbségeket. Ezzel a munka túlmutat egy "játék + AI" demonstráción: egy bővíthető kísérleti platformot ad.

A dolgozat fontos vállalása a **reprodukálhatóság és transzparencia**. A stratégia-összehasonlítás csak akkor értelmezhető, ha a környezet paraméterei rögzítettek, a mérés menete dokumentált, és az eredmények visszaellenőrizhetők. Ezért a rendszerben külön hangsúlyt kap a benchmark eszközlánc, a JSON alapú eredménykimenet, valamint a grafikus összefoglalók előállítás. A fejezetek felépítése ezt a logikát követi: elméleti háttér, architektúra, tervezési folyamat, üzembe helyezés és mérések, felhasználói működés, végül következtetések.

A bevezetőben rögzített célok összhangban állnak a György_Ákos_Szakdolgozat_Specifikáció.docx általános irányával, ugyanakkor a tényleges implementáció több ponton tudatosan pontosított: a kliensoldali játéktér **natív Canvas**

megjelenítést és **TypeScript alapú játéklogikát** használ, a döntéshozatal pedig egységes "állapot be - irány ki" szerződésre épül. Ennek köszönhetően a dolgozat eredménye nem csak egy működő alkalmazás, hanem egy olyan fejleszthető alap, amelyre későbbi kutatási vagy termékjellegű bővítések is biztonságosan építhetők.

2. Elméleti megalapozás és szakirodalmi tanulmány

2.1 Szakirodalmi tanulmány

A szakirodalmi áttekintés három, egymáshoz kapcsolódó témakörre épül: (1) klasszikus keresési és játékstratégiák rácsalapú környezetben, (2) megerősítéses tanulás és neuroevolúciós megközelítések diszkrét akcióterű feladatokra, valamint (3) reprodukálható benchmark- és kiértékelési módszerek. A Snake-játék különösen alkalmas összehasonlító vizsgálatokra, mert egyszerre egyszerű szabályrendszerű és mégis gyorsan növekvő állapottérrel rendelkezik.

Az útkeresés-alapú szakirodalom (A^* , BFS, gráfbejárási módszerek) rámutat, hogy rövid távon optimális lépések és hosszú távú túlélés között folyamatos kompromisszum van. A klasszikus legrövidebb út és heurisztikus keresési alapok a Dijkstra- és A^* cikkekre vezethetők vissza (Dijkstra, 1959; Hart, Nilsson és Raphael, 1968). A kizárólag „étel felé optimalizáló” döntések gyakran késői bezáródáshoz vezetnek, ezért a modern Snake-heurisztikákban megjelenik a biztonsági komponens (fárok-követés, flood-fill alapú szabad tér becslés, ciklikus bejárások). Ez indokolja, hogy a dolgozatban az A^* és BFS mellett Hamilton-jellegű, lookahead és max-safety stratégiák is szerepelnek.

A megerősítéses tanulási irodalom (DQN, PPO) szerint a policy minősége erősen függ a reprezentációtól és a jutalomfüggvénytől; ennek elméleti alapjai a klasszikus RL-irodalomban és a Q-learningben jelennek meg (Sutton és Barto, 2018; Watkins és Dayan, 1992), a mély neurális megközelítés pedig a DQN munkával vált gyakorlati mérőföldkövé (Mnih et al., 2015). A Snake-jellegű környezetekben tipikus probléma a ritka, késleltetett jutalom, ezért a reward shaping és az epizódhossz-korlát döntően befolyásolja a konvergenciát; ez összhangban van a mélytanulás általános reprezentációs és optimalizálási tapasztalataival is (Goodfellow, Bengio és Courville, 2016). A PPO-alapú stabil policy-frissítés különösen releváns ilyen diszkrét akcióterű környezetben (Schulman et al., 2017). A NEAT jellegű neuroevolúciós vonal alternatívát ad, mert nem gradient-alapú tanítást használ, hanem populációs szelekciót és mutációt, így más hibamintázatokat mutat, mint az RL algoritmusok (Stanley és Miikkulainen, 2002).

Az értékelési módszertani források kiemelik, hogy egyetlen mutató (pl. átlagpont) nem elegendő. A medián, szórás, haláloprofil és első cél elérési arány együtt ad megbízható képet. Ennek megfelelően a dolgozat benchmark része többdimenziós metrika-készletet alkalmaz, és a frontend Statisztika oldal is ezt a szemléletet követi.

2.2 Elméleti alapok

Az alkalmazás elméleti alapja egy diszkrét idejű, rácsalapú döntési folyamatként írható le, amely természetesen értelmezhető Markov-döntési folyamat (MDP) keretben (Bellman, 1957; Russell és Norvig, 2020). Egy adott időpillanatban a rendszer állapota tartalmazza a kígyó testének celláit, az aktuális irányt, az étel pozícióját és a pálya geometriáját. A vezérlési feladat célja olyan akciósorozat választása, amely maximalizálja a pontszámot, miközben elkerüli az ütközéseket és az éhezéssel leállást.

A stratégiai döntéshozatal két nagy csoportra oszlik. A heurisztikus ág explicit szabályokra, útkeresésre és lokális biztonsági becslésekre támaszkodik; előnye a kiszámíthatóság és az alacsonyabb „tanítási” igény. A minimax-jellegű komponensek értelmezéséhez az alfa-béta metszés klasszikus elemzése ad támpontot (Knuth és Moore, 1975). A tanult ág (DQN, PPO) és a

neuroevolúciós ág (NEAT) paraméteres modellben reprezentálja a döntési politikát, amely tapasztalatból vagy populációs optimalizációból fejlődik.

A reprezentációs alapelv szerint a játékállapotot a tanuló algoritmusok egy kompakt, normalizált megfigyelésvektoron keresztül kapják meg. Ez egyszerre teszi lehetővé a neurális modellek számára a tanulást és a különböző pályaméretetek közötti részleges általánosítást. A reward függvény kialakítása központi elméleti kérdés: az ételjutalom, túlélési komponens, távolságalapú shaping és az epizódkorlát együtt határozza meg, hogy a modell agresszív pontszerző vagy túl konzervatív túlélő viselkedést tanul.

A kiértékelés elméleti alapja a reprodukálhatóság: azonos seed, azonos pályaméret, azonos lépéslimit és azonos mérőszámok mellett összevethetővé válnak a stratégiák. A dolgozat ennek megfelelően a középértékek mellett eloszlás- és halálok-alapú elemzést is alkalmaz.

2.3 Ismert hasonló alkalmazások

Több olyan nyilvános Snake-AI projekt azonosítható, amelyek jól reprezentálják a fő algoritmikus irányokat, és releváns összehasonlítási alapot adnak a dolgozat számára.

Interaktív neurális hálós/evolúciós demók:

- **NEAT JavaScript példa** (neat-javascript.org/examples/snake.html): valós idejű evolúció, állítható populáció és sebesség, NEAT-alapú hálófejlődés.
- **SnakeAI (genetikus + NN)** (jonatan5524.github.io/SnakeAI/Snake): több kígyós populáció, fitness-alapú szelekció, neurális háló + genetikus algoritmus kombináció.
- **RL Snake (DQN)** (pedro-torres.com/snake-rl): reward alapú online tanulás, Deep Q-Learning megközelítés.

Kísérletezős/sandbox környezet:

- **Snake Lab** (snakelab.osoyalce.com): több modelles család (pl. RNN/CNN jellegű kísérletek), statisztikák és tréning-adatok megjelenítése, kutatásközei felhasználásra.

AI ellen játszható webes példa:

- **Snekgame** (snekgame.io): ember vs AI játékmenet, stratégia-összehasonlítás gameplay környezetben.

A fenti források alapján a szakirodalmi/tájékozódási térkép jól strukturálható algoritmikus családok szerint:

- **evolúciós neurális módszerek** (NEAT),
- **genetikus + neurális hibrid megközelítések**,
- **megerősítéses tanulás** (DQN),
- **sandbox jellegű többmodellű kutatási felületek**,
- **heurisztikus/AI gameplay fókuszú demók**.

Ez a bontás alátámasztja a dolgozat saját összehasonlítási stratégiáját is: heurisztikák, evolúciós módszerek és RL modellek közös, egységes benchmarkolása ugyanazon környezetben.

2.4 Felhasznált technológiák

2.4.1 Frontend technológiák

Programozási nyelvek és formátumok (frontend): TypeScript, JavaScript (build tooling), HTML, CSS.

Alap stack: Vite 5, React 18, React DOM, TypeScript 5.6, HTML5 Canvas.

Fejlesztői és minőségbiztosítási csomagok (frontend/package.json):

- @vitejs/plugin-react (React build integráció Vite-hoz)
- @vitejs/plugin-basic-ssl (helyi HTTPS dev támogatás)
- eslint, @eslint/js, typescript-eslint
- eslint-plugin-react-hooks, eslint-plugin-react-refresh
- @types/react, @types/react-dom, globals

Frontend architektúra-elemek: SPA képernyőkezelés, context alapú állapot (ThemeContext, AuthContext), Canvas renderelés, localStorage perzisztencia, REST + WebSocket kliens kommunikáció.

2.4.2 Backend technológiák

Programozási nyelvek és formátumok (backend): TypeScript, JavaScript futtatókörnyezetben (Node.js), SQL (SQLite DDL/DML), JSON (API szerződések).

Futtatási stack: Node.js, Express 4, TypeScript, tsx (fejlesztői futtatás/watch), tsc (build).

Backend függőségek (backend/package.json):

- express
- better-sqlite3
- bcrypt
- jsonwebtoken
- cors

Backend dev függőségek:

- @types/express, @types/node
- @types/better-sqlite3, @types/bcrypt, @types/jsonwebtoken, @types/cors
- tsx, typescript

Biztonsági és adatkezelési komponensek: JWT Bearer auth, bcrypt jelszóhash, SQLite perzisztencia (users, scores), CORS alapú böngészős API-hozzáférés.

2.4.3 AI service technológiák

Programozási nyelvek és formátumok (AI/benchmark): Python, JSON.

AI szolgáltatás alap stack: Python 3.10+, FastAPI, uvicorn (HTTP + WebSocket kiszolgálás).

AI és tanítási függőségek (ai_service/requirements.txt):

- fastapi
- uvicorn[standard]

- torch (DQN tanítás és)
- numpy
- stable-baselines3 (PPO)
- gym
- neat-python (NEAT / neuroevolúció)

A tanuló moduloknál a fő könyvtári háttér a PyTorch (Paszke et al., 2019) és a Stable-Baselines3 PPO implementációja (Raffin et al., 2021).

Benchmark és vizualizáció: a benchmark futtatás Python scriptekkel történik (`run_benchmark.py`, `run_strategy_benchmark.py`), a grafikon-generálás pedig **matplotlib** csomagot használ (`generate_benchmark_plots.py` szerint opcionálisan telepítendő).

További projekt-szintű eszköz: a gyökérben a `concurrently` NPM csomag biztosítja a három szolgáltatás párhuzamos indítását (`npm run dev:all`).

3. A rendszer specifikációi és architektúrája

3.1 Monorepo, fájlstruktúra

3.1.1 Monorepo áttekintés

Mappa / fájl	Tartalom
frontend/	React + TypeScript + Vite, játéklógika, UI, auth kliens
backend/	Node.js + Express + SQLite, auth, profil, pontszámok
ai_service/	Python FastAPI, WebSocket, stratégiák, SnakeEnv, tanítás
benchmarks/	run_strategy_benchmark.py, generate_benchmark_plots.py, results/, plots/
docs/	Specifikáció, szakdolgozati dokumentáció, egyéb részeredmények
Gyökér package.json	npm run dev, npm run dev:backend, npm run dev:ai, npm run dev:all
README.md (gyökér)	Követelmények, telepítés, futtatás, portok

3.1.2 Frontend fájlstruktúra

A frontend kód a `frontend/src/` alatt funkcionális bontásban szervezett: külön modulok kezelik a játék motorját (`core`), az AI-kliens réteget (`ai`), a futtatási horgokat (`hooks`), a vizuális elemeket (`ui`, `view`) és a perzisztenciát (`io`).

```

frontend/
├── index.html
├── package.json, vite.config.ts, tsconfig.json
├── public/
└── src/
    ├── main.tsx, App.tsx, api.ts, theme.css
    ├── ThemeContext.tsx, AuthContext.tsx, vite-env.d.ts
    ├── core/
    │   ├── board.ts, collision.ts, food.ts, game.ts
    │   └── rng.ts, score.ts, snake.ts, types.ts, index.ts
    ├── ai/
    │   └── Strategy.ts, strategies.ts, strategyBenchmarkDetail.ts
    ├── hooks/
    │   └── useAIWebSocket.ts, useAIGameLoop.ts, useGameLoop.ts
    ├── io/
    │   └── config.ts, storage.ts
    ├── view/
    │   └── GameCanvas.tsx, HUD.tsx
    └── ui/
        ├── Header.tsx, MainMenu.tsx, Settings.tsx
        ├── PlayerSettings.tsx, PlayerSettingsPanel.tsx
        ├── AISettings.tsx, AISettingsPanel.tsx
        ├── LoginForm.tsx, RegisterForm.tsx
        └── Profile.tsx, Results.tsx, Statistics.tsx

```

Ez a felosztás támogatja, hogy a játékmotor (core) minimálisan függjön a React-komponensektől, így a játékállapot logika külön is érthető és újrafelhasználható.

3.1.3 Backend fájlstruktúra

A backend backend/src/ mappában rétegenként szervezett: induló alkalmazás, adatbázis hozzáférés, middleware, és útvonal-szintű üzleti logika.

```

backend/
├── package.json, tsconfig.json
├── data/snake.db
└── src/
    ├── index.ts
    ├── db/
    │   └── sqlite.ts
    ├── middleware/
    │   └── auth.ts
    └── routes/
        ├── auth.ts
        ├── profile.ts
        └── scores.ts

```

Az útvonalbontás közvetlenül követi a funkcionalitást: autentikáció (auth.ts), profilműveletek (profile.ts), eredménykezelés (scores.ts).

3.1.4 AI service fájlstruktúra

Az AI szolgáltatásnál külön kezeljük a futó API réteget (src/main.py), a játékállapot absztrakciót (src/state.py), a környezetet (src/env) és a stratégia-implementációkat (src/strategies). A tanítás és modellfájlok külön könyvtárban vannak.


```

ai_service/
├── README.md, requirements.txt, package.json
├── models/
├── training/
├── src/
│   ├── __init__.py, main.py, state.py
│   ├── env/
│   │   ├── __init__.py, snake_env.py
│   │   └── strategies/
│   │       ├── __init__.py, base.py, rl_stubs.py
│   │       ├── astar.py, bfs.py, follow_tail.py, greedy.py
│   │       ├── hamilton.py, hamilton_zigzag.py, hamilton_short_cycles.py
│   │       ├── lookahead.py, lookahead_n.py, minimax.py, max_safety.py
│   │       └── dqn.py, ppo.py, neat_strategy.py

```

Megjegyzés: futás közben a Python `__pycache__` fájlokat is létrehoz, de ezek nem forráskódok, ezért a dokumentált struktúrában nem szerepelnek.

3.1.5 Benchmark fájlstruktúra

A benchmark réteg a projekt többi részétől elkülönített script- és eredménykészlet. Külön script futtatja a méréseket, külön script készíti a vizualizációt, és a kimenetek JSON/PNG formában a `results` alatt tárolódnak.

```

benchmarks/
├── README.md
├── run_benchmark.py
├── run_strategy_benchmark.py
├── generate_benchmark_plots.py
├── results/
│   ├── benchmark_astar.json
│   ├── benchmark_hamilton.json
│   ├── benchmark_summary.json
│   ├── strategy_benchmark_summary.json
│   ├── strategy_benchmark_*.json
│   └── plots/
│       ├── score_mean_bar.png
│       ├── score_distribution_boxplot.png
│       ├── death_distribution_stacked.png
│       └── score_vs_steps_scatter.png

```

A `run_strategy_benchmark.py` a közös stratégiaregiszterre építve készít összehasonlítható futásokat, míg a `generate_benchmark_plots.py` ezekből szemléltető ábrákat generál.

3.2 Frontend

3.2.1 Állapot és adatfolyam

A frontend architektúra központja az `App.tsx`, amely egyszerre kezeli a képernyőnavigációt, a játékállapotot, a játékmód-választást és az adatforrások (`localStorage` / `backend` / `AI service`) közti váltást. A képernyők egy diszkrét `screen` állapotban reprezentáltak:

```

type Screen = 'menu' | 'settings' | 'results' | 'statistics' | 'game' | 'login' | 'regi

```

A vezérlés két párhuzamos ciklusra bontott:

- **játékos mód:** useGameLoop (setInterval) futtatja a tick függvényt,
- **MI mód:** useAIGameLoop minden tick után WebSocket választ kér, majd irányt állít.

MI adatfolyam lépésről lépésre:

1. tick(prev) előállítja az új lokális állapotot.
2. getSnapshot(newState) serializálható pillanatképet ad.
3. A kliens elküldi a snapshotot a /ws végpontra, opcionális strategy mezővel.
4. A válasz (action) alapján setDirection(...) fut.
5. Canvas újrender a friss állapot alapján.

Ha a WebSocket nem ad időben választ, vagy nincs kapcsolat, a frontend **helyi fallback** stratégiát használ. Ez fontos üzemeltetési döntés: a játék MI módban sem „fagy meg” hálózati hiba esetén.

3.2.2 Adatszerkezetek és stratégia interfész

Frontend oldalon a típusosságot a core/types.ts és a hookok közti szerződések biztosítják. A snapshot mezői (snake, direction, food, rows, cols, seed, tick, score) közvetlenül átadhatók a Python szolgáltatásnak.

A frontend stratégiaválasztó (ai/strategies.ts) explicit listában tartja a támogatott azonosítókat, neveket és rövid leírásokat, így a UI és az AI service regisztere összehangolt marad:

```
export const AI_STRATEGIES = [
  { id: 'astar', name: 'A*', ... },
  { id: 'hamilton', name: 'Hamilton (spirál)', ... },
  { id: 'bfs', name: 'BFS', ... },
  // ...
  { id: 'dqn', name: 'DQN', ... },
  { id: 'ppo', name: 'PPO', ... },
  { id: 'neuroevolution', name: 'NEAT', ... },
] as const
```

WebSocket payloadban a stratégia opcionális mezőként kerül átadásra:

```
const payload: Record<string, unknown> = {
  snake: state.snake,
  direction: state.direction,
  food: state.food,
  rows: state.rows,
  cols: state.cols,
  seed: state.seed,
  tick: state.tick,
  score: state.score,
}
if (strategy !== null) payload.strategy = strategy
ws.send(JSON.stringify(payload))
```

3.2.3 Játékállapot

A játék aktuális állapota a `GameState` típusban van tárolva (pályaméret, kígyó, irány, étel, pont, lépésszám, fázis, seed). Az `App.tsx` induláskor `createGame(config)` hívással hozza létre; futás közben a **játékos módban** a `useGameLoop`, **MI módban** a `useAIGameLoop` frissíti tickenként. A `screen === 'game'` és a `gameMode` dönti el, melyik hurok aktív, így a menü nézetek nem indítanak játéklógikát.

```
const [screen, setScreen] = useState<Screen>('menu')
const [config, setConfig] = useState(loadConfig)
const [gameState, setGameState] = useState<GameState>(() => createGame(config))
const [gameMode, setGameMode] = useState<'player' | 'ai'>('player')

const { aiConnected } = useAIGameLoop(
  gameState,
  setGameState,
  screen === 'game' && gameMode === 'ai',
  config.ai?.strategy ?? 'astar'
)
useGameLoop(
  gameState,
  setGameState,
  screen === 'game' && gameMode === 'player'
)
```

A billentyűkezelés és a játékfázisok explicit állapotgépként működnek (INIT, RUNNING, PAUSED, GAME_OVER). A vezérlésből látszik, hogy a frontend csak érvényes fázisban enged bizonyos műveleteket:

```
if (e.key === 'p' || e.key === 'P') {
  if (gameState.phase === 'RUNNING' || gameState.phase === 'PAUSED') {
    setGameState((s) => (s.phase === 'RUNNING' ? pauseGame(s) : resumeGame(s)))
  }
}
if (e.key === 'Enter' && gameState.phase === 'INIT') {
  setGameState((s) => startGame(s))
}
```

3.2.4 Perzisztencia és UI

A frontend kettős perzisztencia-modellt alkalmaz:

- **lokális tárolás:** `loadConfig/saveConfig`, `loadScores/saveScore` (vendég módhoz),
- **szerver oldali tárolás:** `submitScore / fetchScores` (bejelentkezett felhasználóhoz).

A játék végekor automatikus mentés történik, majd opcionálisan backend szinkron:

```

if (gameState.phase === 'GAME_OVER' && screen === 'game') {
  saveScore({ score: gameState.score, tick: gameState.tick, ... })
  setScores(loadScores())
  if (token) {
    submitScoreApi(gameState.score, gameState.tick, gameState.snakeBody.length, gameMod
  }
}

```

A UI komponensek képernyőnként külön fájlokban vannak (MainMenu, Settings, Results, Statistics, LoginForm, RegisterForm, Profile), de közös fejléctet (Header) és egységes kártya/stílus rendszert használnak. Ez egyszerűsíti az állapotfüggő nézetváltást route-rendszer nélkül.

3.2.5 Téma és auth kontextus

A témakezelés külön contextben történik, és a teljes dokumentumgyökér attribútumát állítja:

```

useEffect(() => {
  document.documentElement.setAttribute('data-theme', theme)
  localStorage.setItem(STORAGE_KEY, theme)
}, [theme])

```

Az autentikációs állapot (AuthContext) token + user párost kezel, perzisztensen:

```

const setAuth = useCallback((t: string, u: User) => {
  localStorage.setItem(TOKEN_KEY, t)
  localStorage.setItem(USER_KEY, JSON.stringify(u))
  setToken(t)
  setUser(u)
}, [])

```

A védett API-hívások közös headerekből épülnek, így a komponenseknek nem kell külön tokenkezelést végezniük:

```

function authHeaders(): HeadersInit {
  const token = getToken()
  return {
    'Content-Type': 'application/json',
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  }
}

```

3.2.6 Statisztika oldal

A `Statistics.tsx` a benchmark-adatokat olvasási és elemzési felületként jeleníti meg, külön élekciklusban a játékhuroktól. A betöltéskor kezeli a hiányzó adatokat és a szolgáltatás hibáit:

```
useEffect(() => {
  let cancelled = false
  fetchBenchmarkSummaries()
    .then((data) => {
      if (cancelled) return
      const list = data.summaries ?? []
      setRows(list)
      setBenchDir(data.benchmark_dir ?? null)
      setVisibleIds(new Set(list.map((r) => r.strategy)))
    })
    .catch((e) => {
      if (!cancelled) setLoadError(e instanceof Error ? e.message : 'Ismeretlen hiba')
    })
  return () => { cancelled = true }
}, [])
```

A táblázat többszintű elemzést támogat: oszloprendezés, stratégiaszűrés (heurisztikus / neurális), sorra kattintható részletpanel, valamint PNG lightbox. Emiatt a frontend nem csak „kliens-játék”, hanem benchmark-kiértékelő dashboard is.

A HUD MI állapotjelzője operatív visszajelzést ad játék közben:

```
{aiConnected !== undefined && (
  <span title={aiConnected ? 'ai_service WebSocket' : 'Helyi stratégia'}>
    MI: <strong>{aiConnected ? `backend (${getStrategyName(aiStrategy)})` : 'helyi'}</strong>
  </span>
)}
```

3.3 Backend

3.3.1 Adatbázis és séma

A backend beágyazott adatbázist használ (better-sqlite3), a fizikai adatfájl a backend/data/snake.db. Az adatbázis-inicializálásánál az első hívásnál létrejön a data mappa, megnyílik az adatbázis, majd lefut a sémaellenőrzés és migráció.

```
function getDb(): Database.Database {
  if (!db) {
    if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true })
    db = new Database(DB_PATH)
    initSchema(db)
  }
  return db
}
```

A séma két fő táblára épül:

- **users:** email, username, jelszóhash, létrehozási idő,
- **scores:** user hivatkozás, pont, tick, hossz, mód (player / ai), opcionális ai_strategy, időbélyeg.

```
CREATE TABLE scores (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL REFERENCES users(id),
  score INTEGER NOT NULL,
  tick INTEGER NOT NULL,
  length INTEGER NOT NULL,
  mode TEXT NOT NULL CHECK (mode IN ('player', 'ai')),
  ai_strategy TEXT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_scores_user ON scores(user_id);
CREATE INDEX IF NOT EXISTS idx_scores_created ON scores(created_at DESC);
```

A `schema_version` tábla verziózott migrációt tesz lehetővé. A jelenlegi implementáció `SCHEMA_VERSION = 2` mellett képes régi `scores` struktúrát újrastrukturálni (`scores_new` létrehozás, másolás, átnevezés). Ez szakdolgozati szempontból fontos, mert demonstrálja, hogy az adatmodell módosítható adatvesztés nélkül.

3.3.2 JWT middleware

A backend tokenalapú hitelesítést használ. A middleware minden védett route előtt ellenőrzi a Bearer formátumot, validálja a JWT-t, majd a payloadot a kérésobjektumhoz csatolja.

```
export function authMiddleware(req: Request, res: Response, next: NextFunction): void {
  const authHeader = req.headers.authorization
  if (!authHeader?.startsWith('Bearer ')) {
    res.status(401).json({ error: 'Hiányzó vagy érvénytelen token' })
    return
  }
  const token = authHeader.slice(7)
  try {
    const payload = jwt.verify(token, JWT_SECRET) as JwtPayload
    ;(req as Request & { user: JwtPayload }).user = payload
    next()
  } catch {
    res.status(401).json({ error: 'Érvénytelen vagy lejárt token' })
  }
}
```

A token tartalma minimális, de elégséges a backend döntésekhez:

- `userId`: adatbázis-kapcsolás és jogosultság,
- `username`: UI-kompatibilis kontextus.

A tokenkiadás külön függvényben történik (`signToken`), 7 napos lejárattal:

```
export function signToken(payload: JwtPayload): string {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' })
}
```

Ez a felépítés biztosítja, hogy a route-logika egyszerű maradjon: a hitelesítés és hibakezelés egy központi rétegben van.

3.3.3 Végpontok és üzleti szabályok

A backend belépési pontja (`index.ts`) három route-csoportot regisztrál:

```
app.use('/api/auth', authRoutes)
app.use('/api/profile', profileRoutes)
app.use('/api/scores', scoresRoutes)
```

Módszer	Útvonal	Leírás
POST	/api/auth/register	regisztráció
POST	/api/auth/login	bejelentkezés
GET/PATCH	/api/profile/me	profil lekérés/módosítás
PATCH	/api/profile/me/password	jelszómódosítás
GET/POST	/api/scores	eredmények lekérés/feltöltés

Auth route-ok (`routes/auth.ts`) üzleti szabályai:

- regisztrációnál email-formátum, username-minta (3–32, betű/szám/_/-), minimum 6 karakteres jelszó, jelszóegyezés,
- egyedi email és egyedi username ellenőrzés,
- jelszó hash-elés (`bcrypt.hashSync(..., 10)`), majd tokenkiadás.

```
if (!EMAIL_REGEX.test(String(email).trim())) { ... }
if (!USERNAME_REGEX.test(String(username).trim())) { ... }
if (String(password).length < 6) { ... }
if (String(password) !== String(passwordConfirm)) { ... }
const password_hash = bcrypt.hashSync(String(password), 10)
```

Login logika: felhasználónév vagy email alapú lookup, majd bcrypt compare. Hibás hitelesítésre egységes 401 válasz.

```
const row = isEmail
  ? db.prepare('SELECT ... FROM users WHERE email = ?').get(input.toLowerCase())
  : db.prepare('SELECT ... FROM users WHERE LOWER(username) = ?').get(input.toLowerCase())
if (!row || !bcrypt.compareSync(String(password), row.password_hash)) {
  res.status(401).json({ error: 'Hibás felhasználónév/e-mail vagy jelszó.' })
  return
}
```

Profile route-ok (`routes/profile.ts`):

- GET /me: aktuális user adatainak lekérdezése token alapján,
- PATCH /me: felhasználónév-csere validációval és ütköztetésvizsgálattal,
- PATCH /me/password: jelenlegi jelszó ellenőrzése, új jelszó szabályok, hash-frissítés.

Scores route-ok (`routes/scores.ts`):

- GET /api/scores: userhez tartozó eredmények, `created_at` DESC, max 200 rekord,

- POST /api/scores: kötelező numerikus mezők (score, tick, length), módnormalizálás (ai vagy player), AI stratégia csak AI módnál menthető.

```
const modeVal = mode === 'ai' ? 'ai' : 'player'
const aiStrategyVal = (modeVal === 'ai' && typeof ai_strategy === 'string' && ai_strategy !== ''
  ? ai_strategy.trim()
  : null
db.prepare(
  'INSERT INTO scores (user_id, score, tick, length, mode, ai_strategy) VALUES (?, ?, ?, ?, ?, ?)'
).run(user.userId, score, tick, length, modeVal, aiStrategyVal)
```

Összességében a backend réteg jól elkülönített felelősségi körökkel működik: index.ts (bootstrap), middleware (auth), db (perzisztencia + migráció), routes (üzleti szabályok). Ez közvetlenül támogatja a frontend által elvárt API-szerződést és a benchmark eredmények felhasználói mentését.

3.4 AI service

3.4.1 Közös interfész és állapot

Az AI service célja, hogy a frontentől érkező játék-pillanatképből (GameStateSnapshot) minden tickben egyetlen irányt adjon vissza (Up, Right, Down, Left). A közös absztrakció a Strategy.next_move(state: GameState) -> Direction, így minden heurisztika és tanult stratégia ugyanarra az interfészre csatlakozik.

A stratégiák több közös építőkövet használnak:

- **állapotreprzentáció:** GameState (head, snake, food, rows, cols, direction, tick, score);
- **szimuláció:** simulate_step egyetlen lépés virtuális futtatására;
- **iránykorlát:** azonnali 180° visszafordulás tiltása (OPPOSITE);
- **gráf-segédek:** astar_path, bfs_path, path_to_tail, direction_from_to;
- **biztonsági metrika:** flood_fill_count és safest_local_step.

A gyakorlatban ez azt jelenti, hogy eltérő döntési filozófiák (útkeresés, cikluskövetés, előretekintés, maximin) összehasonlíthatók ugyanazon állapotmodell és ugyanazon kimeneti szerződés mellett.

3.4.2 Heurisztikus stratégiák (A*, BFS, FollowTail, Hamilton, Lookahead, Minimax)

A felsorolt stratégiák mind ugyanarra a bemeneti állapotra és kimeneti irány-interfészre épülnek, ezért közvetlenül összehasonlíthatók benchmark környezetben.

3.4.2.1 A* (astar)

Az astar_path függvény klasszikus prioritási soros keresést futtat, ahol $f = g + h$, h pedig Manhattan-távolság. Akadály a kígyó teste (fárok nélkül) és a fal.


```
# astar.py
while open_set:
    _, g, current = heapq.heappop(open_set)
    if current == goal:
        ...
    for n in neighbors(current):
        tent_g = g + 1
        ...
        heapq.heappush(open_set, (tent_g + h, tent_g, n))
```

Döntési logika:

1. A* út az ételig.
2. Ha ez nem adható, `path_to_tail` (fárok felé menekülő út).
3. Ha ez sem stabil, `safest_local_step` (max flood-fill szabadságfok).

Ez a stratégia jó baseline, mert egyszerre célorientált és rendelkezik biztonsági visszalépésekkel.

3.4.2.2 BFS

A BFS heurisztika nélkül, rétegenként keresi a legrövidebb lépésszámú utat az ételig. Mivel egységkölségű a rácsmozgás, optimális úthosszt ad, de jellemzően több csomópontot jár be, mint az A*.

Döntési logika:

1. `bfs_path` az ételig.
2. Ha nincs, `path_to_tail`.
3. Ha az sem működik, `safest_local_step`.

*Különbség az A-hoz képest:** a BFS nem használ Manhattan-heurisztikát, ezért „vakabb”, de jól értelmezhető kontrollstratégia.

3.4.2.3 FollowTail

Ez a stratégia hibrid: előbb próbál étel felé haladni (A*), de ha ez kockázatos, farkkövetésre vált, hogy a kígyó önmagának „teret tartson”.

Döntési logika:

1. A* első lépés az étel felé, ha érvényes.
2. `path_to_tail` és annak első lépése, lehetőleg 180° forduló nélkül.
3. `safest_local_step`.

Intuíciója: nem minden áron pontszerzés, hanem túlélési pálya fenntartása, különösen közép- és végjátékban.

3.4.2.4 Hamilton család (spirál, zigzag, short cycles)

Ez a csoport ciklikus pályakövetést ad, így a kígyó determinisztikus mintázatban bejárja a táblát, és kisebb eséllyel zárja magát csapdába.

3.4.2.4.1 Hamilton spirál

A `_build_spiral_cycle` a pályát külső peremtől befelé járja be. Alapértelmezésben a stratégia a ciklus következő cellájára lép (`next_on_cycle`), de étel esetén A* „levágást” enged, ha az első lépés biztonságos.

3.4.2.4.2 Hamilton zigzag

A `_build_zigzag_cycle` soronként váltott irányú bejárást hoz létre (bal->jobb, majd jobb->bal). A logika megegyezik a spirál változattal: cikluskövetés + opcionális A* levágás.

3.4.2.4.3 Hamilton short cycles

A pálya 2x2 blokkokra bontott, minden blokkban egy mini-ciklussal (`block_cycle_cells`, `next_in_block_cycle`). Étél esetén A* első lépés előnyt kap, különben blokkciklus következik, végül safety fallback.

3.4.2.5 Greedy

A legegyszerűbb biztonságorientált stratégia: minden tickben `safest_local_step`, ami a legnagyobb flood-fill elérhető teret választja.

```
class GreedyStrategy(Strategy):
    def next_move(self, state: GameState) -> Direction:
        return safest_local_step(state)
```

Erősség: stabil túlélés bizonyos helyzetekben.

Gyengeség: kevés explicit ételcélzás, ezért könnyen éhezéshedomináns.

3.4.2.6 Max Safety

Ez a stratégia egy extra biztonsági feltételt ad: csak olyan lépést enged, amely után továbbra is van út a fej és a fark között (`has_path_head_to_tail`).

Döntési logika:

1. minden érvényes, nem 180°-os lépés szimulálása;
2. fej-fark összeköttetés ellenőrzése;
3. a megmaradt jelöltek közül max flood-fill.

Intuíciója: a kígyó ne vágja el saját „kijáratát”.

3.4.2.7 Lookahead 1

Egy lépéses előretekintés: minden lehetséges irányra kiszámítja a szabadságfokot (`flood_fill_count`), döntetlenben az ételtávolság dönt.

```

if count > best_count or (count == best_count and dist < best_dist):
    best_count = count
    best_dist = dist
    best_d = d

```

Ez gyakran jó kompromisszum: olcsó számítás + célorientált tie-break.

3.4.2.8 Lookahead N

A stratégia több lépésre előre szimulál (`simulate_greedy_n`), ahol a belső döntések greedy értékelést használnak. Az `evaluate` függvény kombinálja a szabadságfokot és az ételtávolságot:

érték = `flood_fill_count - 0.5 * manhattan_distance`.

lookahead_3: rövidebb horizont, gyorsabb.

lookahead_5: mélyebb horizont, de drágább és bizonyos konfigurációkban túl konzervatív.

3.4.2.9 Minimax

A minimax implementáció 1-2 lépéses horizontot használ. Minden első lépésre megnézi a második lépések legrosszabb esetét, majd a maximin elv szerint választ.

```

for d1 in DIRECTIONS:
    ...
    val = 10.0**9
    for d2 in DIRECTIONS:
        ...
        val = min(val, evaluate(s2))
    if val > best_value:
        best_value = val
        best_first = d1

```

Ez konzervatív stratégia: nem átlagos, hanem „legrosszabb várható” folytatások ellen próbál robusztus maradni.

3.4.3 Tanult stratégiák (DQN, PPO, NEAT)

A heurisztikus stratégiákkal szemben a tanult megközelítések **előre betanított modellekből** döntenek: a bemenet mindig ugyanaz a **12 dimenziós** megfigyelésvektor (`state_to_observation`), a kimenet pedig négy diszkrét irány egyike (Up, Right, Down, Left). A három implementáció ugyanarra a **Strategy** interfészre épül (`next_move(state) -> Direction`), és a szolgáltatás **STRATEGIES** regiszterében külön azonosítóval érhető el:

```

STRATEGIES = {
    # ... heurisztikák ...
    "dqn": DQNStrategy,
    "ppo": PPOStrategy,
    "neuroevolution": NEATStrategy,
}

```

A frontend vagy a benchmark a stratégia nevét (`dqn`, `ppo`, `neuroevolution`) küldi; a `main.py` **get_strategy(name)** példányosítja a megfelelő osztályt, majd minden lépésnél meghívja a

next_move metódust (REST POST /next vagy WebSocket /ws).

Közös bemenet – megfigyelés. A `GameState` $\rightarrow$ `list[float]` konverzió a `snake_env.py`-ban történik; a vektor tartalma megegyezik a tanítási környezet (SnakeEnv) bemenetével (részletes táblázat: [3.4.6.1](#)):

```
def state_to_observation(state: GameState) -> list[float]:
    # 12 float: veszély 4 irányban, fal távolság, étel relatív pozíció/távolság, kígyóh
    ...
    assert len(obs) == OBSERVATION_DIM # OBSERVATION_DIM = 12
    return obs
```

Közös szabály – 180° tiltás. Snake szabály szerint a kígyó nem fordulhat azonnal vissza; mindhárom tanult stratégia ezt **maszkolással** érvényesíti: az ellentétes irány indexe „ $-\infty$ ” pontszámot kap, így nem kerülhet kiválasztásra.

Közös fallback. Ha hiányzik a modellfájl, a PyTorch / Stable-Baselines3 / neat-python könyvtár, vagy a betöltés hibázik, a stratégia **nem áll le**, hanem a **GreedyStrategy** (biztonság-orientált heurisztika) lép helyette. Benchmarkon és webes játékban ez azt jelenti, hogy a stratégia **neve** alatt is **más viselkedés** mérhető, ha a modell nincs telepítve – reprodukálhatósághoz a `models/` tartalmát rögzíteni kell.

A **tanítás** folyamata (replay buffer, PPO klip, NEAT populáció) a [3.4.6](#) alfejezetben van; itt az **inferencia** (döntés pillanatában) részletezése a cél.

3.4.3.1 DQN (dqn)

A **Deep Q-Network** megközelítés egy MLP hálóval becsüli a **Q(s, a)** értékeket mind a négy akcióra; inferencia közben a legnagyobb Q indexét választja iránynak (**greedy policy** a tanult Q mellett).

Modellfájlok: `ai_service/models/dqn_snake.pt` (súlyok) és `dqn_snake_config.json` (pl. `obs_dim`, `hidden`, `n_actions`). Opcionális környezeti változó: **DQN_MODEL_DIR**.

Betöltés és háló: a `dqn.py` lazy importtal tölti a PyTorch-ot; a config alapján építi fel az MLP-t (alapértelmezés: $12 \rightarrow 128 \rightarrow \text{ReLU} \rightarrow 128 \rightarrow \text{ReLU} \rightarrow 4$).

Döntés egy lépésben:

```
obs = state_to_observation(state)
with t.no_grad():
    x = t.tensor([obs], dtype=t.float32)
    q = self._model(x).clone()
    cur_idx = DIR_TO_ACTION.get(state.direction, 1)
    invalid = opposite_action_index(cur_idx)
    q[0, invalid] = -1e9
    action = int(q.argmax(dim=1).item())
    return ACTION_NAMES[action]
```

A tanítás során használt **ϵ -greedy** felfedezés inferencia közben **nem** fut; a benchmark és a webes MI mód **determinisztikus** (legjobb Q). A tanító script opcionális **Double DQN** logikát is tartalmaz; a tanítás részletei a [3.4.6.2](#) fejezetben találhatók.

3.4.3.2 PPO (ppo)

A **Proximal Policy Optimization** itt nem saját implementáció, hanem a **Stable-Baselines3** PPO osztályán keresztül fut: a betanított policy közvetlenül ad diszkrét akcióindexet 0–3 között.

Modellfájlok: először `models/ppo_snake_best.zip`, ha nincs, akkor `models/ppo_snake.zip`.

Betöltés és döntés:

```
def _load_ppo_model(model_dir: str | None = None):
    candidates = [
        os.path.join(model_dir, "ppo_snake_best.zip"),
        os.path.join(model_dir, "ppo_snake.zip"),
    ]
    path = next((p for p in candidates if os.path.isfile(p)), None)
    ...
    return PPOClass.load(path)

# next_move:
obs = np.array(state_to_observation(state), dtype=np.float32)
action, _ = self._model.predict(obs, deterministic=True)
a = int(action)
# ellentétes irány → Greedy fallback, egyébként ACTION_NAMES[a]
```

A **deterministic=True** kiszűri a tanítás közbeni sztochasztikust, így ugyanazon állapothól ugyanaz az akció jön ki. A PPO a Gym-kompatibilis becsomagolt SnakeEnv-en tanult; lásd a [3.4.6.3](#) fejezetben a tanítás részleteit.

3.4.3.3 NEAT (neuroevolution)

A **Neuroevolution of Augmenting Topologies** evolúcióval állít elő neurális hálót; inferencia közben egy **feedforward** háló aktiválja a bemenetet, a négy kimenet közül pedig az **argmax** választ irányt.

Modellfájl: `models/neat_snake_best.pkl` – pickle tartalom: **genome** + **config_path** (NEAT konfigurációs fájl útvonala).

Betöltés és döntés:

```
config = neat.Config(neat.DefaultGenome, neat.DefaultReproduction, ...)
net = neat.nn.FeedForwardNetwork.create(genome, config)

obs = state_to_observation(state)
out = self._net.activate(obs)
scores = list(out)
scores[invalid] = -1e9 # 180° tiltás
action_idx = int(max(range(len(scores)), key=lambda i: scores[i]))
return ACTION_NAMES[action_idx]
```

A regiszterben az azonosító **neuroevolution**, az osztály neve **NEATStrategy** (`neat_strategy.py`). A tanítás a `training/train_neat.py` scripttel történik; lásd a [3.4.6.4](#) fejezetben.

3.4.3.4 Összefoglaló összehasonlítás

Azonosító	Osztály	Modell / könyvtár	Kimenet	Fallback
dqn	DQNStrategy	PyTorch .pt + .json	$Q \rightarrow \text{argmax}$	Greedy
ppo	PPOStrategy	SB3 .zip	predict $\rightarrow$ akció	Greedy
neuroevolution	NEATStrategy	neat-python + .pkl	activate $\rightarrow$ argmax	Greedy

3.4.4 REST és WebSocket viselkedés

- **GET /health**, **GET /strategies** – stratégia lista / állapot.
- **POST /next** – egy lépés JSON állapotból.
- **WebSocket /ws** – streamelt állapot $\rightarrow$ válasz: **action** (irány).
- **GET /benchmark/summaries**, **GET /benchmark/plots/{filename}** – a [6.11 Statisztika oldal](#) szerinti statisztika nézet táplálása; a szervernek a **benchmarks/results/** és **plots/** elérhetőnek kell lennie a fájlrendszerben.

3.4.5 Stratégiák táblázatban

Azonosító	Osztály / modul	Rövid logika
astar	AStarStrategy	A* ételig $\rightarrow$ farok / legbiztonságosabb
bfs	BFSStrategy	BFS ételig $\rightarrow$ ugyanaz a fallback
follow_tail	FollowTailStrategy	A* étel $\rightarrow$ farok $\rightarrow$ safest
greedy	GreedyStrategy	Max flood fill
hamilton	HamiltonianStrategy	Spirál ciklus + A* levágás
hamilton_zigzag	HamiltonianZigzagStrategy	Zigzag ciklus + A* levágás
hamilton_short_cycles	HamiltonShortCyclesStrategy	2×2 blokkciklus + A*
lookahead	LookAheadStrategy	1 lépés: szabadság + étel
lookahead_3 / lookahead_5	LookAheadNStrategy	N lépés greedy szimuláció
minimax	MinimaxStrategy(depth=2)	2 lépés maximin
max_safety	MaxSafetyStrategy	Fej-farok út + max flood

Azonosító	Osztály / modul	Rövid logika
dqn	DQNStrategy	Q-háló
ppo	PPOStrategy	SB3 PPO
neuroevolution	NEATStrategy	NEAT háló

3.4.6 Tanuló környezet és algoritmusok (SnakeEnv, DQN, PPO, NEAT)

Ez a fejezet a **tanításhoz és a tanult stratégiák bemenetéhez** kötődő **SnakeEnv** környezetet és a **DQN / PPO / NEAT** algoritmusok **tanítási logikáját** foglalja össze. A [3.4.3](#) a **döntés pillanatában** futó stratégiákat írja le; itt az a kérdés, **hogyan készül** a tanult Q-háló, PPO policy vagy NEAT genom.

A három tanító script közös alapja a **SnakeEnv** (`ai_service/src/env/snake_env.py`), amely a **GameState** és a **simulate_step** függvényt használja – ugyanazt a fizikát, mint a játék és a benchmark. A tanítás kimenete a **ai_service/models/** könyvtárba kerül; az AI szolgáltatás ezeket tölti be inferencia közben ([3.4.3](#)).

Algoritmus	Script	Kimenet
DQN	<code>training/train_dqn.py</code>	<code>dqn_snake.pt</code> , <code>dqn_snake_config.json</code>
PPO	<code>training/train_ppo.py</code>	<code>ppo_snake_best.zip</code> (és kompat: <code>ppo_snake.zip</code>)
NEAT	<code>training/train_neat.py</code>	<code>neat_snake_best.pkl</code> (genom + config útvonal)

3.4.6.1 SnakeEnv

Fájl: `ai_service/src/env/snake_env.py`. **Interfész:** `reset(seed=None) -> (observation, info), step(action: int) -> (observation, reward, done, info)`. **Akciók:** 0=Up, 1=Right, 2=Down, 3=Left (`ACTION_NAMES`, `DIR_TO_ACTION`). A környezet **nem** teljes Gym implementáció, de ugyanazt a `reset / step` szerződést követi; a PPO tanításhoz egy vékony **SnakeGymEnv** wrapper adja a `gym.Env` interfészt ([3.4.6.3](#)).

Megfigyelés. A **state_to_observation** függvény állítja elő a **12 float** vektort (`OBSERVATION_DIM = 12`), amely megegyezik a [3.4.3](#) inferenciánál használt bemenettel:

Index	Tartalom
0–3	Veszély (Up, Right, Down, Left): 1.0, ha fal vagy test 1–2 lépésben
4–7	Fal távolság négy irányban, normalizálva <code>max(rows, cols)</code> szerint
8–9	Étel relatív: $(\text{food.x} - \text{head.x})/\text{cols}$, $(\text{food.y} - \text{head.y})/\text{rows}$; nincs étel $\rightarrow 0,0$
10	Ételtől Manhattan / $(\text{rows} + \text{cols})$; nincs étel $\rightarrow 1.0$

Index	Tartalom
11	Kígyóhossz / (rows*cols)

A veszélydetektálás és a fal-távolság számítása irányonként történik:

```
OBSERVATION_DIM = 12

for d in ACTION_NAMES:
    dx, dy = DELTA[d]
    danger = 0.0
    for step in range(1, 3):
        nx, ny = head[0] + dx * step, head[1] + dy * step
        if not (0 <= nx < cols and 0 <= ny < rows) or (nx, ny) in body:
            danger = 1.0
            break
    obs.append(danger)
```

Jutalom. A konstruktor paramétereizhető; alapértelmezés: étel **reward_food**, halál **reward_death**, túlélés **reward_survival** minden lépésben, Manhattan **reward_step_toward** / **reward_step_away** ha nem evett. A **max_steps_per_episode** elérésekor **done=True** és **reward_starvation** jár (epizód időkorlát):

```
def __init__(self, rows: int = 20, cols: int = 20, reward_food: float = 1.0,
              reward_death: float = -10.0, reward_step_toward: float = 0.03,
              reward_step_away: float = -0.03, reward_survival: float = 0.02,
              reward_starvation: float = -0.5,
              max_steps_per_episode: int | None = None):
    ...
```

Egy **step** hívás logikája: **simulate_step** → halál esetén azonnali **reward_death**; étkezésnél **reward_food**; egyébként túlélés + opcionális távolság-alapú shaping; étkezés után új étel spawnol a környezet saját **Random** generatorával (seed → reprodukálhatóság):

```
raw_new = simulate_step(self._state, direction)
if raw_new is None:
    return obs, self.reward_death, True, {"done_reason": "death"}

ate = raw_new.food is None and food is not None
reward = self.reward_food if ate else 0.0
reward += self.reward_survival

if raw_new.food is None:
    new_food = _random_empty_cell(self.rows, self.cols, occupied, self._rng)
    ...

if self.max_steps_per_episode is not None and self._step_count >= self.max_steps_per_episode:
    done = True
    reward += self.reward_starvation
```

Reset. A kígyó középen indul, három cellával, irány **Jobbra**; az első étel üres cellára kerül:


```

cx, cy = self.cols // 2, self.rows // 2
snake = [(cx - 1, cy), (cx - 2, cy), (cx - 3, cy)]
food = _random_empty_cell(self.rows, self.cols, occupied, self._rng)
self._state = GameState(snake=snake, direction="Right", food=food, ...)

```

3.4.6.2 DQN tanítás

Ötlet. Egy MLP neurális háló becsüli a $Q(s,a)$ értékeket mind a négy irányra. Tanítás közben ϵ -greedy politika (felfedezés $\rightarrow$ kihasználás), a cél a **TD-hiba** minimalizálása egy **replay buffer** mintáiból; a **target network** periodikus másolása stabilizálja a tanulást. Opcionálisan **Double DQN**: a következő akciót a policy háló választja, a Q-értéket a target háló adja (van Hasselt, Guez és Silver, 2016). Optimalizálás: **Adam** (Kingma és Ba, 2015).

Háló és buffer. training/train_dqn.py:

```

class DQN(nn.Module):
    def __init__(self, obs_dim: int, n_actions: int, hidden: tuple[int, ...] = (128, 128),
        ...
        layers.append(nn.Linear(prev, n_actions))
        self.net = nn.Sequential(*layers)

class ReplayBuffer:
    def push(self, obs, action, reward, next_obs, done):
        self.obs[self.pos] = obs
        ...
        self.pos = (self.pos + 1) % self.capacity

```

Tanítási ciklus. Minden lépésnél: (1) **180° tiltás** – az ellentétes irány indexe kiesik az ϵ -greedy és a greedy választásból; (2) tapasztalat a bufferbe; (3) ha elég minta van, batch frissítés; (4) target háló szinkronizálás **target_update_every** lépésenként:

```

invalid_action = opposite_action_index(env.current_direction_index())
if np.random.random() < epsilon:
    action = int(env._rng.choice(valid_actions))
else:
    q = policy_net(t).clone()
    q[0, invalid_action] = -1e9
    action = int(q.argmax(dim=1).item())

next_obs, reward, done, info = env.step(action)
buffer.push(obs_arr, action, reward, next_arr, done)

if use_double_dqn:
    next_actions = policy_net(no).argmax(1, keepdim=True)
    next_q = target_net(no).gather(1, next_actions).squeeze(1)
else:
    next_q = target_net(no).max(1)[0]
target = r + gamma * next_q * (1 - d)
loss = nn.functional.mse_loss(q, target)

```

Hiperparaméterek és mentés. Alapértelmezett rács: **20×20**, **25 000** epizód; gamma=0.99, batch_size=64, buffer_size=50_000, ϵ : 1.0 $\rightarrow$ 0.05. Opcionális **--curriculum**:

először 12×12 (10k ep), majd 20×20 (15k ep) a korábbi súlyokkal. A legjobb modell az utolsó **100 epizód** átlagos jutalma alapján kerül mentésre:

```
if mean_100 > best_mean_reward:
    torch.save(policy_net.state_dict(), "models/dqn_snake_best.pt")
    torch.save(policy_net.state_dict(), "models/dqn_snake.pt") # inferencia
    json.dump({"obs_dim": 12, "n_actions": 4, "hidden": [128, 128]}, ...)
```

Lásd a [3.4.3.1](#) fejezetben az inferencia részleteit.

3.4.6.3 PPO tanítás

Ötlet. Közvetlenül **stochastikus politika** $\pi(a|s)$ tanítása (diszkrét: négy akció), **value függvény** $V(s)$ a **GAE** / advantage becsléséhez; **klipelt** policy loss, hogy egy frissítés ne legyen túl nagy (Raffin et al., 2021). A projekt a **Stable-Baselines3** PPO implementációját használja, nem saját backprop ciklust.

Gym wrapper és jutalom-preset. A SnakeEnv becsomagolása SnakeGymEnv-ként adja a Box observation space-t és a Discrete(4) action space-t. A tanítás **ételorientált presetet** használ (méréselt shaping, időkorlát), hogy a policy ne csak „túléljen”, hanem aktívan keresse az ételt:

```
FOOD_PRESET = {
    "reward_food": 10.0,
    "reward_survival": 0.0,
    "reward_step_toward": 0.05,
    "reward_step_away": -0.05,
    "reward_starvation": -0.5,
    "max_steps_per_episode": 2000,
}

PPO_ENT_COEF = 0.01

class SnakeGymEnv(gym.Env):
    def __init__(self, rows=20, cols=20, use_food_preset=True):
        self.env = SnakeEnv(rows=rows, cols=cols, **FOOD_PRESET) if use_food_preset else SnakeEnv(...)
        self.observation_space = spaces.Box(..., shape=(OBSERVATION_DIM,), dtype=np.float32)
        self.action_space = spaces.Discrete(len(ACTION_NAMES))

    def step(self, action: int):
        obs, reward, done, info = self.env.step(int(action))
        return np.array(obs, dtype=np.float32), float(reward), done, False, info
```

Tanítás és mentés. DummyVecEnv + PPO("MlpPolicy", vec_env, ent_coef=0.01); alapértelmezett **800 000** timestep 20×20 pályán. Értékelés után a legjobb modell kerül a ppo_snake_best.zip-be (kompat: ppo_snake.zip):

```
vec_env = DummyVecEnv([make_env()])
model = PPO("MlpPolicy", vec_env, verbose=1, seed=seed, ent_coef=PPO_ENT_COEF)
model.learn(total_timesteps=800_000)
model.save("models/ppo_snake_last.zip")

# evaluate_ppo → ha jobb mean_reward, akkor:
model.save("models/ppo_snake_best.zip")
```

Lásd a [3.4.3.2](#) fejezetben az inferencia részleteit.

3.4.6.4 NEAT tanítás

Ötlet. Egy **populáció genomjai** (súlyok, opcionálisan topológia) **fitness** szerint szelektálódnak; nincs klasszikus RL backprop – a **neat-python** könyvtár végzi a mutációt, reprodukciót és fajképzést. A bemenet ugyanaz a 12 dimenziós vektor; a kimenet **4 neurális aktiváció**, **argmax** választ irányít (tanítás közben nincs 180° maszk – ez inferenciánál kerül be, [3.4.3.3](#)).

Környezet és fitness. A NEAT-hez enyhén ételorientált, időkorlátos környezet:

```
def make_env(rows, cols):
    return SnakeEnv(rows=rows, cols=cols, reward_food=1.0,
                    reward_survival=0.01, reward_starvation=-0.5,
                    max_steps_per_episode=2000)
```

Egy genom értékelése: feedforward háló aktiválása → lépések összes jutalma + **score** × 10 bónusz (hogy a tényleges pontszám is befolyásolja a fitness-t):

```
def eval_genome(genome, config, rows, cols, episodes=5):
    net = neat.nn.FeedForwardNetwork.create(genome, config)
    for ep in range(episodes):
        env = make_env(rows, cols)
        obs, _ = env.reset(seed=42 + ep)
        while not done and steps < 5000:
            out = net.activate(obs)
            action_idx = int(np.argmax(out))
            obs, reward, done, info = env.step(action_idx)
            total_fitness += reward
        total_fitness += float(info.get("score", 0)) * 10.0
    return total_fitness / episodes
```

Populáció és mentés. Alapértelmezett config: 150 egyed, 12 bemenet, 4 kimenet, `initial_connection = full_direct`. A `pop.run` hívja az `eval_genomes`-t generációnként; a legjobb genom csak akkor íródik felül, ha jobb fitness-et ér el, mint a korábbi best:

```
config = neat.Config(neat.DefaultGenome, neat.DefaultReproduction,
                    neat.DefaultSpeciesSet, neat.DefaultStagnation, config_path)
pop = neat.Population(config)
winner = pop.run(lambda genomes, cfg: eval_genomes(genomes, cfg, rows, cols), n=generat

if winner_fitness > prev_best_fitness:
    pickle.dump({"genome": winner, "config_path": config_path}, f)
```

Lásd a [3.4.3.3](#) fejezetben az inferencia részleteit.

3.5 Benchmark

3.5.1 Benchmark szerepe az architektúrában

A benchmark réteg a rendszer mérési „igazságforrása”: ugyanazt a `GameState` reprezentációt és ugyanazt a `STRATEGIES` regisztert futtatja, amit az AI service játék közben használ. Emiatt az összehasonlítás nem eltérő implementációk között, hanem ugyanazon döntési logikák között történik kontrollált protokollban.

Architektúrális szerepek:

- **szolgáltatásfüggetlen mérés:** a benchmark nem a frontend loopból mér, hanem offline reprodukálható szkripttel;
- **regiszteralapú bővíthetőség:** új stratégia automatikusan benchmarkolható, ha bekerül a STRATEGIES mappingbe;
- **egységes kimenet:** minden stratégia azonos JSON szerkezetet kap (summary + runs);
- **UI-integráció:** a frontend Statistics oldal közvetlenül ezeket a fájlokat fogyasztja AI service /benchmark/* végpontokon át.

A belépési pont a run_strategy_benchmark.py, amely explicit az ai_service csomagra fűzi fel magát:

```
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from ai_service.src.state import GameState, DELTA, OPPOSITE
from ai_service.src.strategies import STRATEGIES
```

A futtató egy körön belül is konzisztens védelmeket használ: 180° tiltás korrekció, éhezési cutoff, max lépésszám limit.

A jelen fejezet a benchmark **architektúrális** elhelyezkedését és adatútját rögzíti; a futtatási protokoll, a JSON kimenet, a grafikonok és az eredmények szakmai értelmezése a [5. Üzembe helyezés és kísérleti eredmények](#) következik (különösen [5.2 Kísérleti eredmények, mérések](#)); a felhasználói megjelenítés a [6.11 Statisztika oldal](#) alatt található.

3.5.2 Adatkapcsolat a rendszer többi részével

Az adatút több lépésben épül fel:

1. **Bemenet:** futtatási paraméterek (--runs, --rows, --cols, --max-steps, --seed-base, --strategies).
2. **Stratégia példányosítás:** strategy = STRATEGIES[strategy_key]() (osztály vagy lambda gyár is támogatott).
3. **Szimuláció:** run_one_game(...) minden seedre ugyanazon játékszabályokkal.
4. **Aggregálás:** átlag/medián/halállok/első étel százalék számítás.
5. **Persistálás:** strategy_benchmark_<id>.json + strategy_benchmark_summary.json.
6. **Kiszolgálás:** AI service benchmark-végpontok.
7. **Vizualizáció:** frontend Statistics.tsx táblázat, szűrés, rendezés, lightbox.

Kulcs logika a futtatóban:

```
direction = strategy.next_move(state)
if direction == OPPOSITE.get(state.direction):
    direction = state.direction
...
if steps_since_food >= starvation_limit:
    death = "starvation"
    game_over = True
```

Összegző metrikák előállítása:

```
summary = {
    "score_mean": sum(scores) / len(scores) if scores else 0,
    "score_median": sorted(scores)[len(scores) // 2] if scores else 0,
    "steps_mean": sum(steps_list) / len(steps_list) if steps_list else 0,
    "death_counts": deaths,
    "reached_first_food": sum(1 for s in scores if s >= 1) / len(scores) * 100 if scores
}
```

Kapcsolódó benchmark fájlok:

- benchmarks/run_strategy_benchmark.py (fő, egységes futtató),
- benchmarks/run_benchmark.py (korábbi A* vs Hamilton futtató),
- benchmarks/generate_benchmark_plots.py (PNG grafikonok),
- benchmarks/results/strategy_benchmark_*.json,
- benchmarks/results/plots/*.png.

4. Tervezés folyamata

4.1 Első tervezési fázis – alkalmazásmag és integrációs alapok

Az első fázis célja egy működő, végponttól végpontig használható alaprendszer létrehozása volt, amelyre később tanuló algoritmusok és mérési infrastruktúra is biztonságosan építhető. A commit-sorrend alapján ebbe a szakaszba tartozott: frontend implementálása, AI service alapjainak kialakítása, backend fejlesztése, frontend bővítése backend-integrációval, további AI stratégiák implementálása, majd a játék- és beállítási felület javítása. Az alábbi alpontok **logikailag** ezt az építési sort követik; az üzemeltetési megkönnyítések (pl. egy parancs mindhárom szolgáltatásra) csak **azután** kerülnek szóba, hogy mindegyik komponens önállóan is létezik és indítható.

Frontend implementálása:

Első lépésként elkészült a React + TypeScript alapú kliens, benne a játékmag (core) és a vizuális réteg (ui, view) elkülönítésével. Meghatározásra került a játék fázisállapot-gépe (INIT, RUNNING, PAUSED, GAME_OVER), a billentyűvezérlés, a Canvas renderelés, valamint a képernyőalapú SPA navigáció (menu, settings, game, results stb.). A kezdeti döntés az volt, hogy a játékszabályok tiszta függvényekben maradjanak, így a megjelenítés és a logika szétválik.

4.1.1 Játék újraindítása játék vége után

A játék vége (GAME_OVER) után az „Új játék” gomb korai változata csak `createGame()`-et hívott; a screen React állapot ekkor is `game` maradt, ezért a `useEffect`, amely a játék képernyőre lépéskor indította el a `startGame()`-et, **nem futott újra**. Az állapot INIT-en ragadt, a játék nem váltott RUNNING fázisba (üres vagy beragadt játéknézet). **Megoldás:** új játék indításakor közvetlenül `setGameState(startGame(createGame(...)))` (vagy ezzel egyenértékű hívás), így az új kör azonnal futó játékállapotba kerül, képernyőváltástól függetlenül.

AI service alapjai:

A Python/FastAPI szolgáltatás kezdetben minimális, de stabil szerződéssel indult: `GET /health`, `GET /strategies`, `POST /next` és `WebSocket /ws`. A közös célinterfész (state in -> action out) már ekkor rögzítve lett, ami később lehetővé tette, hogy új heurisztikák és tanult modellek ugyanabba a csővezetékbe kerüljenek.

4.1.2 Frontend és Python közös állapot- és döntési szerződés

A kliens JSON-ben küldi a játék pillanatnyi állapotát; a Python oldal **parse_state** (vagy ezzel egyenértékű) feldolgozással építi a közös `GameState` modellt. Az irányok (Up, Right, Down, Left) és a rácsbeli lépésvektorok mindkét végponton **azonos konvenció** szerint értelmezendők. A **180°-os azonnali visszafordulás** tiltása egységes követelmény: a kliens játékmagja, a benchmark futtató és a stratégiai döntés ugyanazt a szabályt alkalmazza; különben a szerver olyan irányt javasolhat, amit a játék elvet. Ennek megfelelően a benchmark bizonyos esetekben, ha a stratégia szembefordulást adna, **megtartja az aktuális irányt** – így a mérés nem generál szabálysértő lépést.

Backend megvalósítása:

A Node.js + Express + SQLite backend ebben a fázisban kapta meg az autentikációs és perzisztencia-réteget (users, scores), a JWT hitelesítést, valamint a szükséges route-okat (`/api/auth`, `/api/profile`, `/api/scores`). A tervezési cél itt az volt, hogy a játék működjön

vendég módban is, de bejelentkezett felhasználóknál szerver oldali eredménytárolás is rendelkezésre álljon.

Adatbázis-séma bővítése: A `scores` tábla szerkezetének későbbi bővítése kompatibilitási kockázatot hozott. **Megoldás:** verziózott migráció (`schema_version`), amely meglévő adatok mellett is biztonságosan frissít.

Frontend bővítése backend-integrációval:

Az API-klienst (`api.ts`) és az auth kontextust (`AuthContext`) hozzáigazítottuk a backend végpontjaihoz. A bejelentkezés/regisztráció, profilműveletek és score-feltöltés olyan módon került beépítésre, hogy a UI képernyők ugyanazzal az állapotgéppel működjenek, mint korábban, csak adatforrás-szinten bővüljenek (`localStorage` + backend).

Tokenes API-hívások: Lejárt vagy hibás JWT esetén a profil- és score-végpontok 401-es választ adnak. **Megoldás:** egységes auth middleware a backenden, valamint kliens oldali logout és újra-autentikációs folyamat.

4.1.3 Regisztráció és REST: „Failed to fetch”

A regisztrációs űrlap beküldésekor előfordult a böngésző „**Failed to fetch**” hibája. Ez gyakran nem klasszikus HTTP 4xx/5xx válasz, hanem azt jelzi, hogy a **kérés egyáltalán nem jutott el a szerverhez** (például a backend nem fut a várt `localhost:3000` porton). A fejlesztői környezetben a **CORS** megfelelő beállítása a backenden szükséges ahhoz, hogy más originről (pl. a Vite dev szerverről) érkező kérések engedélyezettek legyenek. Hibakereséskor a `GET /health` ellenőrzése és – ha mindhárom komponens már létezik – a párhuzamos indítás ([4.1.5](#)) adott megbízható alapot.

4.1.4 Heurisztikus stratégiák bővítése

Az alap **A*** és **BFS** baseline-ok, a **FollowTail**, a **Hamilton**-variánsok és a **Greedy** réteg után több hullámban kerültek be további **heurisztikus** stratégiák, hogy a benchmark összehasonlítható baseline-kínálata bővüljön: több lépésre előre tekintő **lookahead** ($N = 3-5$), **minimax** rövid horizonton, önálló **max_safety** (fej–farok útbiztonság és flood-fill alapú lokális döntés), **Hamilton rövid ciklusok** (kis blokk-körök kombinációja) stb. A többlépéses előrejelzéshez a `simulate_step` (vagy ezzel egyenértékű) **lépésszimuláció** volt szükséges. Minden új modul a közös **STRATEGIES** regiszterbe került; a kulcsdöntés változatlanul az volt, hogy minden stratégia ugyanarra a `GameState` reprezentációra és azonos iránykimeneti konvencióra támaszkodjon.

Játék- és beállításjavítások:

Finomhangolásra került a `Settings` képernyő, a player/AI mód közötti váltás, a fallback viselkedés, a HUD visszajelzés, valamint a konfiguráció perzisztencia. **Szolgáltatás-elérhetőség MI módban:** ha az AI WebSocket nem elérhető, a játék nem állhat le. **Megoldás:** kliens oldali fallback stratégia (`placeholderStrategy`) és HUD állapot-visszajelzés (backend vs helyi). Ezzel lezárult az alapalkalmazás „termékesítési” szintje: a rendszer már játszható, menthető, autentikálható és AI-val vezérelhető volt.

4.1.5 Három szolgáltatás párhuzamos indítása (`npm run dev:all`)

Időrendi helye: miután a frontend (Vite), a Node backend és az AI service (FastAPI/Uvicorn) mindegyike önálló fejlesztői parancsal elindíthatóvá vált, vált érdekessé a mindennapi munkában a **három külön terminál** helyett egy közös indító parancs. Fejlesztés közben a szolgáltatások tipikusan a `:5173`, `:3000` és `:8000` portokon hallgatnak. **Megoldás:** a monorepo gyökerében a `concurrently` csomag összekapcsolja a három fejlesztői scriptet; egyetlen parancs, a `npm run`

dev:all, egyszerre indítja a szolgáltatásokat, a konzolkimeneten színezett prefixekkel ([frontend], [backend], [ai]) jelezve a forrást.

Az első fázis végére megszületett az a stabil monorepo alap, amelyen a második fázisban a tanuló algoritmusok, a harmadikban pedig a mérési-elemzési réteg felépíthető lett.

4.2 Második tervezési fázis – tanuló algoritmusok fejlesztése

A második fázis fókuszja a tanuló ág teljes kiépítése volt: DQN implementálás és tanítás, további DQN kísérletek, PPO implementálás és tanítás, valamint NEAT implementálás és tanítás. Ebben a szakaszban a rendszer célja már nem csak a működés, hanem a tanuló módszerek összehasonlítható teljesítményének elérése volt.

4.2.1 DQN: megfigyelés-egyeztetés és tanítási érzékenység

A DQN pipeline a **környezet-kompatibilis** állapotvektorra, **replay** alapú tanításra, .pt modellmentésre és inferenciaoldali betöltésre épült. **Követelmény:** a tanítás és az online döntés **ugyanazt a megfigyelés-teret** (state_to_observation / ekvivalens vektor) használja; ha a kettő eltér, a modell élesben **instabillá** válhat („tanult mást, mint amit látsz”).

A korai próbálkozások gyakran **gyenge pontkonverziót** és **stagnáló** politikát mutattak. A következő iterációkban a **reward** alakítása, az **exploráció** (pl. epsilon), az **epizódhossz** és az **éhezéshez kötött** büntetések finomhangolása következett. Összegzés: a DQN ebben a feladatban **érzékeny** ezekre a hiperparaméterekre; kis változás is más halálmintázatot és átlagpontszámot eredményezhet – ez indokolja a dokumentált benchmark melletti óvatos következtetést is.

4.2.2 PPO: környezet, reward finomhangolás és legjobb modell mentése

A PPO ág a **Stable-Baselines3** könyvtárra és egy vékony Gym-wrapperre (SnakeGymEnv) épült, amely a meglévő **SnakeEnv** reset / step szerződését adja át a policy-gradient tanításnak. A **SnakeEnv** bővült többek között **max_steps_per_episode** korláttal és **éhezéshez kötött** negatív jutalommal (starvation), hogy a tanítás ne vesszen el végtelen vagy csak toporgó játszmákban.

Egy **agresszív reward-preset** kísérlet (igen erős ételjutalom, erős éhezés-büntetés, extrém entropia-együttható) **rossz eval-profil**t adott (a dokumentált mérések környezetében az **átlagpontszám** nagyjából **0,30** körülire esett vissza). **Megoldás:** visszahúzás **mérsékelt** értékekre – például reward_food = 10, starvation = -0.5, max_steps = 2000 (epizódonként), ent_coef = 0.01 –, hogy a policy ne „szétfusszon” és összevethetőbb legyen a heurisztikus baseline-okkal.

A tanító script **globális „legjobb”** modellkezelést vezetett be: új futás csak akkor írja felül a ppo_snake_best.zip (és kompatibilitási ppo_snake.zip) fájlokat, ha az értékelés szerinti **jobb** teljesítményt hoz; metaadat (ppo_snake_best_meta.json) és **időbélyeges backup** könyvtár (models/backups_ppo/...) védi a korábbi jó modelleket. Az inferenciában a PPOStrategy ezt a best (vagy utolsó) csomagot tölti; hiány vagy SB3-függőség híján **Greedy fallback** maradt – ez benchmark-szempontról is rögzítendő, nehogy a stratégia neve alatt más viselkedést mérjünk.

4.2.3 NEAT implementálása és tanítása

A harmadik tanuló módszerként a neuroevolúciós ág (NEAT) került integrálásra. Ez tervezési szempontból tudatos diverzifikáció volt: a projekt így nem egyetlen tanulási paradigmára épül, hanem RL (DQN/PPO) és evolúciós megközelítést is tartalmaz. A legjobb egyed mentése és visszatöltése lehetővé tette, hogy NEAT is ugyanabban a benchmark környezetben fusson.

Ennek a fázisnak az eredménye egy teljes tanuló portfólió lett, amely már technikailag összehasonlítható a heurisztikákkal. A hangsúly áttevődött az „implementáció kész” állapotról a „mérhetően értékelhető” állapotra.

4.3 Harmadik tervezési fázis – benchmark, statisztika és következtetések

A harmadik fázis a mérési infrastruktúra és az elemzési réteg felépítésére koncentrált: benchmarkok és statisztika oldal implementálása, majd az eredmények kiértékelése és a következtetések levonása.

Benchmarkok implementálása:

Kialakult az egységes benchmark futtató (`run_strategy_benchmark.py`), amely kontrollált seed-sorozaton, azonos pályaméreten és limitfeltételek mellett futtatja a stratégiákat. A futásonkénti adatok és összesítők JSON formátumba kerülnek, így reprodukálhatók és utólag elemezhetők. Fejlesztés közben **benchmark futások elhúzódása** („körbejárás”) is felmerült: egyes stratégiák kevés pontnövekedés mellett hosszú ideig élnek. **Megoldás:** starvation cutoff (`steps_since_food >= 400`) és fix `max_steps` limit.

Statisztika oldal implementálása:

A frontend Statistics nézet benchmark-fogyasztó dashboarddó vált: táblázatos összegzés, stratégiaszűrés, rendezés, részletpanel és PNG-grafikon megjelenítés. Ez tervezési szempontból lezárta a „mérés -> tárolás -> vizualizáció” adatút vonalat. A dashboard bevezetésekor **vizualizációs hiányok** is előfordultak: üres táblázat vagy hiányzó grafikonok, ha a JSON/PNG nincs legenerálva. **Megoldás:** benchmark és plot script kötelező futtatása, majd az AI service `/benchmark/*` végpontok ellenőrzése.

Eredmények kiértékelése:

A harmadik fázisban a hangsúly már nem új funkciók bevezetésén, hanem az értelmezésen volt: mean/median külön olvasata, halálprofilok, first-food arány, heurisztikák és tanult stratégiák viszonyának módszertani elemzése.

Következtetések:

A projekt végső szakmai állítása ebben a fázisban kristályosodott ki: a vizsgált konfigurációban a heurisztikák stabil baseline-t adnak, míg a tanuló modellek teljesítménye erősen függ a tanítási költségvetéstől és a reward/reprezentációs beállításoktól. Ez nem általános algoritmus-rangsor, hanem rögzített protokoll melletti összehasonlítás.

Ezzel a harmadik fázis lezárta a teljes fejlesztési ívet: kezdeti alkalmazásmag -> tanuló algoritmusok integrálása -> reprodukálható benchmark és szakdolgozati szintű kiértékelés.

5. Üzembe helyezés és kísérleti eredmények

Ez a fejezet a **mérési protokollt** és az **eszközláncot** rögzíti – összhangban a `benchmarks/run_strategy_benchmark.py` forrásával és a [6.11 Statisztika oldal](#) által feltételezett adatstruktúrával.

5.1 Üzembe helyezési lépések

Az üzembe helyezés célja, hogy a három fő komponens (frontend, backend, AI service) egységesen, reprodukálható konfigurációval induljon, és a benchmark-folyamat is végigfuttatható legyen.

Ajánlott lépéssor (helyi fejlesztői környezet):

1. **Követelmények ellenőrzése:** Node.js (LTS), npm, Python 3.10+, pip.
2. **Függőségek telepítése:**
 - gyökér: `npm install` (concurrently a `dev:all` scripthez),
 - frontend/backend: saját `package.json` függőségek,
 - AI service: `pip install -r ai_service/requirements.txt`.
3. **Környezeti változók ellenőrzése:** backend (JWT_SECRET), frontend (VITE_API_URL, VITE_AI_HTTP_URL, VITE_AI_WS_URL), portok ütközésének kizárása.
4. **Szolgáltatások indítása:**
 - külön: `npm run dev`, `npm run dev:backend`, `npm run dev:ai`, vagy
 - együtt: `npm run dev:all`.
5. **Health check és alap funkcionális teszt:**
 - backend: `GET /health`,
 - AI service: `GET /health`, `GET /strategies`,
 - frontend: menü betöltés, játék indítás player és AI módban.
6. **Adatút validálása:** regisztráció/login -> játék -> score mentés -> eredmények oldal -> statisztika oldal.
7. **Benchmark és ábrák generálása:** `run_strategy_benchmark.py`, majd `generate_benchmark_plots.py`; kimenetek ellenőrzése a `benchmarks/results/` mappában.

A forráskód és a kiegészítő anyagok elérését a [9. Függelék](#) foglalja össze; a teljes projekt nyilvános GitHub tárolója: https://github.com/GyorgyAkos/snake_projekt.

5.2 Kísérleti eredmények, mérések

5.2.1 Cél és szerep

A benchmark célja **objektív, reprodukálható összehasonlítás** több stratégia között: ugyanaz a pálya, ugyanaz a kezdőállapot-függő **seed-sorozat**, ugyanaz a lépés- és éhezési limit. Nem „ember elleni verseny”, hanem **döntési logikák** (heurisztika vs. betanított modell) viselkedése **statisztikai mintán**.

5.2.2 Futtató: `run_strategy_benchmark.py`

Belépési pont: projekt gyökeréből érdemes futtatni (a script a `ROOT`-ot a `benchmarks/` szülőjére állítja, és az `ai_service` csomagot teszi a `sys.path`-ra).

Egy játék (`run_one_game`):

1. **STRATEGIES[`strategy_key`]** példányosítása (osztály vagy lambda $\rightarrow$ mindkettő hívható konstruktorként).
2. **Kezdőállapot:** `create_initial_state(rows, cols, seed)` – középen **3** cellás kígyó, irány **Jobbra**, étel véletlen üres cellán ugyanazzal a **seed**-del inicializált **Random**-gal.
3. **Ciklus:** minden lépésben `strategy.next_move(state)`; ha a stratégia a **180°-os** fordulatot adná, a futtató **megtartja az aktuális irányt** (ütközés elkerülése a specifikus edge case-re).
4. **step:** fal / önmaga $\rightarrow$ `game_over + wall` vagy `self`; étel evés $\rightarrow$ növekvő score, új étel.
5. **Éhezés (starvation):** ha `steps_since_food >= 400` (a forrásban fix konstans: „*20×20 pályán 400 lépés érdemi éhezési limit*”), a kör **starvation** okkal leáll – így a végtelen „toporgás” nem nyújtja a benchmarkot.
6. **Lépéslimit:** ha eléri a `max_steps` értéket ütközés nélkül, a halál oka a JSON-ban tipikusan `max_steps` (a kód: `death or "max_steps"`).

CLI paraméterek (alapértelmezés): `--runs 200, --rows 20, --cols 20, --max-steps 5000, --seed-base 42, --strategies` vesszős lista; ha **--strategies** nincs megadva, egy beépített alapkészlet fut (`greedy, max_safety, hamilton, astar, dqn, ppo, neuroevolution`). **Kimeneti könyvtár:** `--out-dir` (alapból `benchmarks/results/`).

Parancs példa:

```
python benchmarks/run_strategy_benchmark.py --runs 500 --strategies greedy,hamilton,max
python benchmarks/run_strategy_benchmark.py --runs 200 --strategies dqn,ppo,neuroevolut
```

5.2.3 Kimenet: JSON és összefoglaló

Cél: a benchmark kimenete gépi és emberi fogyasztásra egyaránt alkalmas legyen: stratégiánként **részletes futáslista** (reprodukálhatóság, utólagos elemzés), valamint egy **összegező blokk** (`summary`), amelyből a [6.11 Statisztika oldal](#) ugyanazokat a mutatókat rajzolja, mint a táblázat oszlopai. A webes nézet szöveges „Metodika” blokkjával összhangban az alábbiakban a **JSON-mezők értelmezése** és a **felületen látható címkék** egymáshoz rendelése is szerepel.

Stratégiánkénti	teljes	fájl:
<code>benchmarks/results/strategy_benchmark_<azonosító>.json</code>		(például
<code>strategy_benchmark_astar.json</code>).	Gyökérszinten két kulcs:	

- **summary:** aggregált metrikák és protokoll-paraméterek (ez olvasható ki közvetlenül a statisztika táblázatba).
- **runs:** lista, elemenként **egy lejátszott kör** eredménye.

Egy `runs[ ]` elem mezői (`run_one_game` kimenete): `score` (végső pontszám), `steps` (lépések száma a leállásig), `death` (leállás oka: `wall, self, starvation`, vagy ütközés nélküli futáslimit esetén `max_steps`), `seed` (a körhöz használt mag: `seed_base + i` a futásindex szerint). Ezekből a **summary** számolódik a futtatóban: például `death_counts` a `death` értékek

gyakorisága, `score_mean` / `score_median` a pontszámokból, `steps_mean` / `steps_median` a lépésszámokból.

A **summary** blokk mezői (megegyeznek a frontend `BenchmarkSummary` típusával):

Mező	Jelentés	Statisztika oldallal való egyezés
<code>strategy</code>	A STRATEGIES kulcs (pl. <code>astar</code>)	A táblázatban a név alatti monó sor (<code>stats-mono</code>).
<code>runs</code>	Futások száma	„Futás” oszlop. A UI szövege szerint nagyobb érték stabilabb átlagot , de hosszabb mérést jelent.
<code>rows, cols</code>	Pályaméret	A részletes panelban: „pálya: <code>rows×cols</code> ” (alapértelmezésben <code>20×20</code>).
<code>max_steps</code>	Lépés korlát futásonként	A részletes panelban szerepel; a „Lépéslimit %” oszlop a <code>max_steps</code> halálok arányát mutatja.
<code>seed_base</code>	Első mag	A részletes panel „mag” mezője; a futások magjai ettől lépnek (<code>+0</code> , <code>+1</code> , ...).
<code>score_mean, score_median</code>	Pont átlaga és mediánja	„Átlag pont” és „Medián”. A web megkülönbözteti őket: az átlag érzékeny a szélső értékekre , a medián a „ tipikus ” kört jelzi.
<code>steps_mean, steps_median</code>	Lépések átlaga és mediánja	A táblázat az átlag lépést (<code>steps_mean</code>) mutatja; a medián a fájlban ott van elemzéshez, de nem külön oszlop a felületen. A Statisztika oldal szerint a hosszabb játék gyakran (de nem mindig) jobb stratégiát vagy óvatosabb túlélést jelent.
<code>death_counts</code>	Halálok darabszáma okonként	A „Fal %”, „Önmaga %”, „Éhség %”, „Lépéslimit %” oszlopok ebből és <code>runs</code> -ból számolódnak százalékra ($100 * \text{darab} / \text{runs}$).
<code>reached_first_food</code>	A futások hány százalékában score ≥ 1 (legalább egy étel)	„1. étel %”. Ha alacsony , a stratégia sokszor „elakad” anélkül, hogy stabilan haladna az étel felé – ugyanez áll a Statisztika oldal szövegében.

A **summary** mező tartalmazza: `strategy`, `runs`, `rows`, `cols`, `max_steps`, `seed_base`, **`score_mean`**, **`score_median`**, **`steps_mean`**, **`steps_median`**, **`death_counts`** (kulcsonkénti darabszám), **`reached_first_food`** (százalék: futások hány százalékában **score** ≥ 1 , azaz legalább egy étel).

A futás végén felülírt fájl: **`strategy_benchmark_summary.json`** – stratégiánként csak a **summary** objektumok (a webes `/benchmark/summaries` ebből vagy a külön JSON-ekből

épít).

5.2.4 Vizualizáció: `generate_benchmark_plots.py`

Futtatás: `python benchmarks/generate_benchmark_plots.py (matplotlib szükséges).` Beolvassa az összes `strategy_benchmark_*.json`-t (a `strategy_benchmark_summary.json` kizárva), és a `benchmarks/results/plots/` mappába menti többek között:

Fájl	Értelmezés
<code>score_mean_bar.png</code>	Gyors rangsor átlagpont szerint
<code>score_distribution_boxplot.png</code>	Stabilitás, szórás, outlierok
<code>death_distribution_stacked.png</code>	Halálok megoszlása (<code>wall / self / starvation / max_steps</code>)
<code>score_vs_steps_scatter.png</code>	Pont vs. lépésszám trade-off

Ezek a fájlok a [6.11 Statisztika oldal](#) nézetén is megjeleníthetők (`/benchmark/plots/...`).

5.3 Eredmények részletes értelmezése

Az alábbi táblázatok egy konkrét, dokumentált kampány számai; a repó `benchmarks/results/*.json` fájljai a **frissíthető forrás**. **Összhangban** állnak a frontend **Statisztika** nézet felhasználói magyarázataival is ([6.11 Statisztika oldal](#)); **ettől függetlenül** az alábbi szöveg **összefoglaló, szakdolgozati** megfogalmazás, és **nem** ismétli végig az alkalmazásbeli szekciókat. A konkrét számok mindig a lokálisan betöltött JSON-tól függenek; stratégiánkénti **általános mintákat** a `strategyBenchmarkDetail.ts` modul is rögzít.

5.3.1 Mit jelentenek a metrikák?

- **runs:** mintanagyság – nagyobb érték **stabilabb átlag**, hosszabb futási idő.
- **Átlag vs. medián pont:** az **átlag** érzékeny a szélsőértékekre; a **medián** a „tipikus” játékot jól jelzi – rangsoroláskor mindkettőt érdemes nézni (**kerülendő az „átlagfetisizmus”**, amikor csak a `score_mean` alapján rangsorolunk).
- **Átlag lépés (`steps_mean`):** a hosszabb játék **gyakran (de nem mindig)** jobb túlélést vagy óvatosabb viselkedést jelez; **önmagában** nem rangsor-szabály.
- **Halál okok (`wall, self, starvation, max_steps`):** a futtató szerinti besorolás; összehasonlító táblázatban gyakran **Fal %**, **Önmaga %**, **Éhség %**, **Lépéslimit %** formában jelennek meg. Az **éhség (`starvation`)** itt **nem** valós játékbeli ütközés: ha hosszú ideig **nem nőtt a pontszám**, a futtató leállította a kört.
- **Hiányzó halál-kulcs a JSON-ban:** ha egy ok egy futásban sem következett be, a kulcs gyakran hiányzik — megjelenítéskor ilyenkor **0%** adódik (nem „hiányzó adat”).
- **Első étel elérése (`reached_first_food`, **százalék**):** ha alacsony, a stratégia sok futásban **nem jut el stabilan az első ételig** („elakadás”), ami rontja a robusztusságot.

5.3.2 Heurisztikák vs. tanult stratégiák

- A **heurisztikák** explicit szabályokat követnek (út keresés, előretekintés, fark-követés stb.), ezért ebben a protokollban gyakran **magasabb átlagpontot** adnak **rövid távon**, ha a szabály jól illeszkedik a pályamérethez.
- A **tanuló** (RL / evolúciós) rendszerek sokszor **gyengébben indulnak** ugyanazon összehasonlításban: a jutalom és az **állapotleképezés** finomhangolása nélkül könnyen kialakul „**biztonságos toporgás**” vagy instabil viselkedés — ez sok **éhség** jellegű leállásban látszik. A dqn, ppo, neuroevolution azonosítók **tanult** stratégiák a benchmarkban.
- **Methodológiai megjegyzés: rosszabb benchmarkeredmény nem jelenti automatikusan, hogy a módszer rossz** — csupán hogy **ebben a környezetben / beállításban** még nem versenyképes. A heurisztikák jó **bázisvonalnak (baseline)** számítanak: ha egy tanult ügynök nem közelíti meg őket, érdemes először a **jutalmat**, az **episód hosszát** és a **megfigyelést** finomítani.

5.3.3 Halálprofilok és szakmai értelmezés

A dokumentált kampány halálprofilja és a [5.3.7](#) szerinti **halálmegoszlás-ábra** együttes olvasmánya:

- **Greedy / MaxSafety / DQN** (ebben a mérésben): gyakorlatilag **csak starvation** – nem feltétlen „jó túlélés”; inkább azt jelzi, hogy a lépések **nem konvertálódnak** hatékonyan pontszerzésre (**alacsony pont + domináló éhség** mintázat).
- **PPO / NEAT**: domináns **starvation**, de már jelen van **self / wall** is → **aktívabb**, kockázatosabb mozgás.
- **A*, BFS, FollowTail, Hamilton, Lookahead**: főként **self** és **wall**; **starvation** elhanyagolható → a stratégiák **aktívan keresik az ételt**, és ütközésben halnak meg inkább, mint éhen — ez gyakran **magas pont** és **alacsony éhségarány** mellett értelmes kép.

Összefüggés: magas **self** / fal arány gyakran **túl kockázatos** döntést vagy **rövid távú nyereséget** jelezhet a biztonság rovására; magas **self** egyúttal **agresszívebb ételkereséssel** is társulhat — ezért a halálprofil **mindig a pontszámmal és az 1. étel %-kal együtt** kell olvasni.

5.3.4 Hatékonyság és robusztusság (összehasonlító keret)

Az összehasonlítás javasolt kerete az összefoglaló metrikák és a grafikonok **együttes** használatával:

- **Hatékonyság:** **score_mean / score_median** és **1. étel %**; az oszlopdiagram gyors összképet ad, a **boxplot** a **stabilitást** és azt, hogy egy stratégia futásai mennyire **szétszórtak** (vannak-e korán elvérző kövek mellett kiugróan jók is).
- **Robusztusság:** magas **reached_first_food** + magas medián pont + nem torz halálmegoszlás.
- **Trade-off:** a **pont vs. átlag lépésszám** szórásvázlat: ki szerzi „**drágán**” a pontot (sok lépés, alacsony score).

A fenti szempontokhoz illeszkedő generált ábrák a [5.3.7 Vizualizáció: generált grafikonok](#) alfejezetben láthatók.

5.3.5 Heurisztikák (500 futás / stratégia, dokumentált kampány)

A táblázat **500 futás / stratégia** mellett készült kampányt mutatja; a **strategy_benchmark_*.json** fájlokból ugyanezek az aggregátumok számíthatók. A nagyobb

minta az átlagokat és a halálarányokat **stabilabbá** teszi, mint kevesebb körben. Értelmezéskor érdemes **éhség- és első étel** szerint is rendezni a sorokat, nem csak átlagpont szerint.

Stratégia	Átlag pont	Medián	Átlag lépés	Első étel (%)
lookahead	92.75	92	2460.71	100.0
astar	77.20	77	1297.47	100.0
follow_tail	77.20	77	1297.47	100.0
bfs	74.40	76	1245.71	100.0
hamilton	57.01	57	851.08	100.0
lookahead_3	10.93	10	180.68	100.0
minimax	4.41	3	446.56	80.6
lookahead_5	0.30	0	403.73	15.2
greedy	0.20	0	404.62	15.0
max_safety	0.20	0	404.62	15.0

Tipikus mintázatok a számokból. Magas pont és relatíve alacsony éhség együttesen erős étel- és pontszerzési aktivitásra utal — ez **lookahead**, **astar**, **follow_tail**, **bfs**, **hamilton** és **lookahead_3** esetén áll fenn (mind **100%** első étel, a minimax kivételével **80,6%**). Ellenoldalon **greedy**, **max_safety** és **lookahead_5** extrém alacsony átlag- és mediánpont mellett **nagyon alacsony 1. étel %-ot** mutat (**alacsony pont + domináló éhség**): *beragadás*, nem elég **célvezérelt** viselkedés. A `strategyBenchmarkDetail` modul ezt a működési logikához köti: **greedy** nem célvezérelt az étel felé, **max_safety** extrém konzervatív, **lookahead_5** esetén a mélyebb szimuláció **nem mindig javít**, és gyenge átlag / magas éhség előfordulhat.

Csoportos összkép (strategyBenchmarkDetail alapján):

- **Legerősebb sor: 1 lépéses lookahead** — *lokálisan jól kerüli a zsákutcát, és mégis cél orientált*; a táblán **legmagasabb** átlag és medián, hosszú átlagos lépésszám: tartós pontszerzés.
- **Útvonalkereső felső mezőny: A*, follow_tail, BFS** — erős étel felé vezetés és **biztonsági tartalék**; A* és BFS eredményávja gyakran közel áll egymáshoz.
- **Strukturált bejárás: Hamilton** — kiszámítható túlélés, de az étel felé rövidítés nem mindig optimális; **kisebb** átlagpont, de **100%** első étel.
- **Mélyebb lookahead: lookahead_3** még **100%** első étellel, de **alacsonyabb** átlag, mint 1 lépésnél — a *mélyebb előretekintés* önmagában nem garantál jobb átlagot. **Lookahead_5** a modul és a táblázat szerint is **lesüllyedhet**: a szimuláció **torzíthat** vagy túl óvossá teheti a politikát.
- **Minimax**: rövid horizont → változékony döntés; **1. étel % 80,6**, alacsony pont — tipikusan **nem éri el** a BFS / A* szintet ebben a mérésben.

Összehasonlítás a tanult stratégiákkal: a [5.3.6](#) táblázat **200 futásra** készült; ugyanazon skálán való következtetéshez az [5.2.3](#) fejezetben említett **azonos runs** kontrollfutás ajánlott. A repóban a kampányok **független** futtatásokból is állhatnak össze.

5.3.6 Tanuló stratégiák (200 futás / stratégia)

Ez a blokk **200 futás / stratégia** mellett készült — **nem** azonos mintanagyság, mint az [5.3.5 500 futásos](#) heurisztikai táblázat. Szűkítő következtetéseknél (két stratégia egyetlen számmal történő „megverése”) tehát a **runs különbségét** is figyelembe kell venni; az összefoglaló JSON **runs** mezője ezt dokumentálja.

Stratégia	Átlag pont	Medián	Átlag lépés	Első étel (%)
neuroevolution (NEAT)	7.38	5	1027.16	90.5
ppo	2.08	2	384.67	66.0
dqn	0.17	0	403.36	12.5

Sorrend és robusztusság: NEAT > PPO > DQN mindhárom fő mutatóban. A NEAT **90,5%-os 1. étel %-a** azt jelzi, hogy sok futásban **eléri az első ételt** és mozdul a pontnövekedés irányába; a PPO **66%-kal hátrébb** van, a DQN **12,5%-kal erős elakadás**-mintát mutat — sok körben **nem** jut el stabilan az első ételig, ami illusztrálja a **strategyBenchmarkDetail** DQN-magyarázatát (policy könnyen *topog*, magas éhség és alacsony pont, ha a jutalom / állapot / tanítási büdzsé nem megfelelő).

Működési logika és tipikus következmények (strategyBenchmarkDetail):

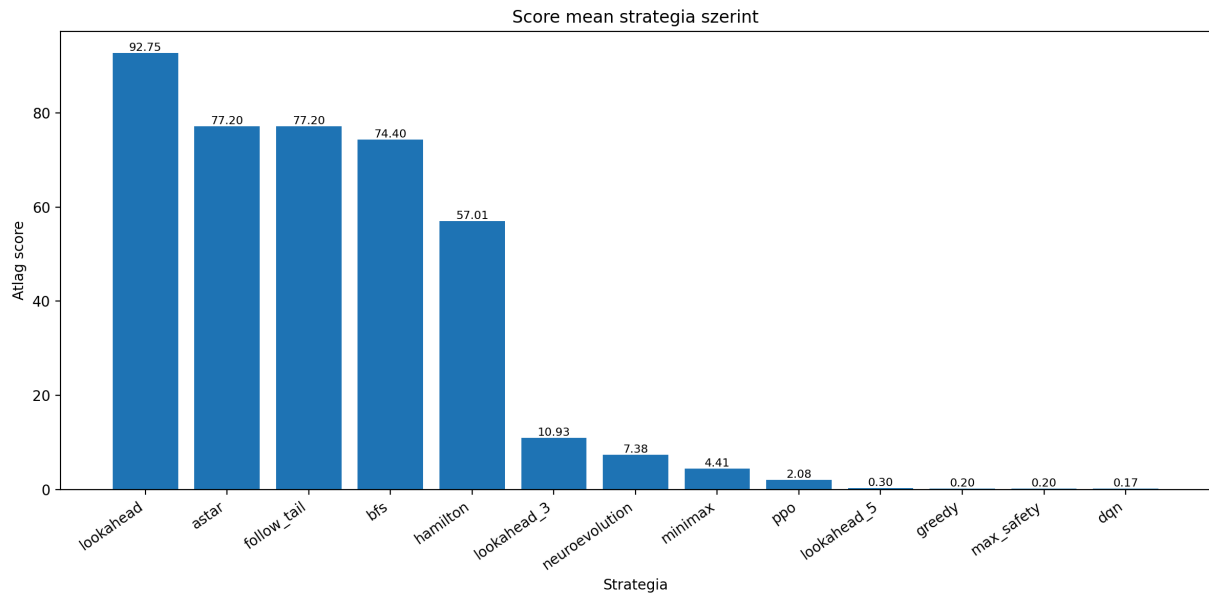
- **DQN:** $Q(s,a)$ becslés; rossz jutalom-shaping vagy állapotleképezés **éhség-domináns** viselkedéshez vezethet, miközben **1. étel %** nagyon alacsony maradhat. **Nem** következtethetünk általános „*alkalmatlan algoritmus*” állításra: a dokumentált benchmark csak azt mondja ki, hogy **ez a konfiguráció** nem versenyképes a **heurisztikus baseline**-hoz képest.
- **PPO:** sztochasztikus **policy**; hasonlóan érzékeny a jutalomra és az epizód hosszára. Itt az átlagos lépés **rövidebb, éhségben** gyakran végződő kövek felé hajlik inkább, mint a NEAT; **1. étel % közepes** sáv.
- **NEAT:** populációs evolúció; **fitness** és **bemeneti feature-ök** határozzák meg az „érdemes-e aktívan gyűjteni” viselkedést. Ez a mérés **távol** esik a top heuristikáktól, de **felül** van a másik két tanult stratégián — a tanuló módszerek induló teljesítménye változó; **egy számjegy** nem minősíti el őket minden környezetben.

Összkép: a [5.3.2](#) és [5.3.3](#) szerinti baseline-logika **konkrét számokban** jelenik meg: a heuristikák többsége az 5.3.5 táblában **nagyságrenddel magasabb** pontot ad; javítási irány: **jutalom, epizód hossz, megfigyelés** / feature finomhangolás, rögzített modellfájlok ([3.4.3 Tanult stratégiák](#)).

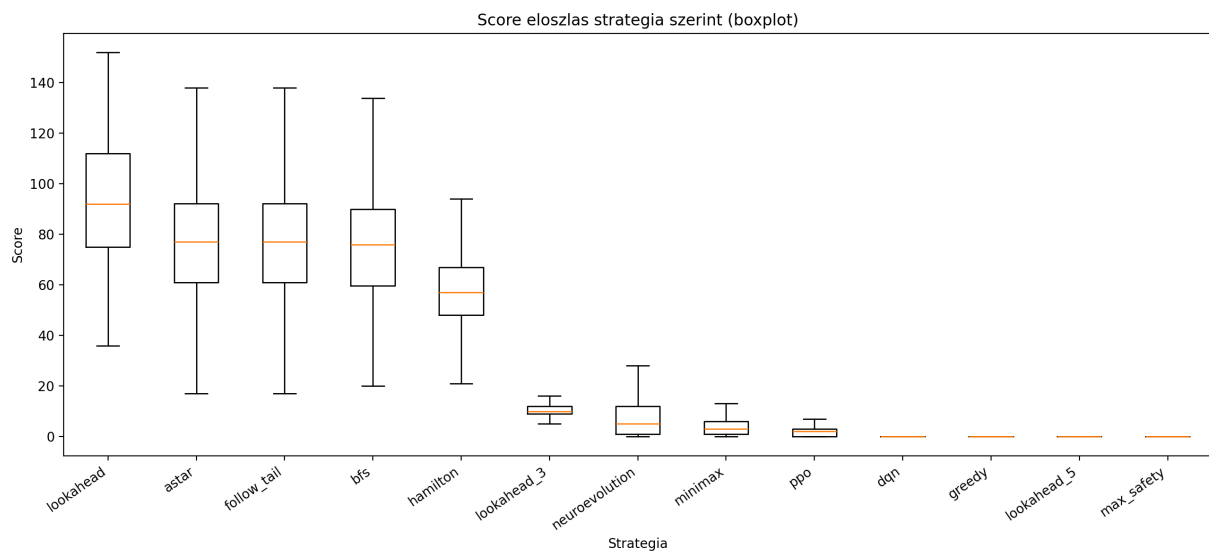
5.3.7 Vizualizáció: generált grafikonok (PNG)

Az alábbi ábrák a **benchmarks/generate_benchmark_plots.py** szkripttel készülnek a **benchmarks/results/strategy_benchmark_*.json** fájlokból; a kimenet a **benchmarks/results/plots/** mappába kerül. Ugyanezekhez a fájlokhoz a futtató mellett a kifejlesztett kliens-AI szolgáltatás réteg is hozzáférhet (HTTP-n keresztül, a benchmark plot végponton).

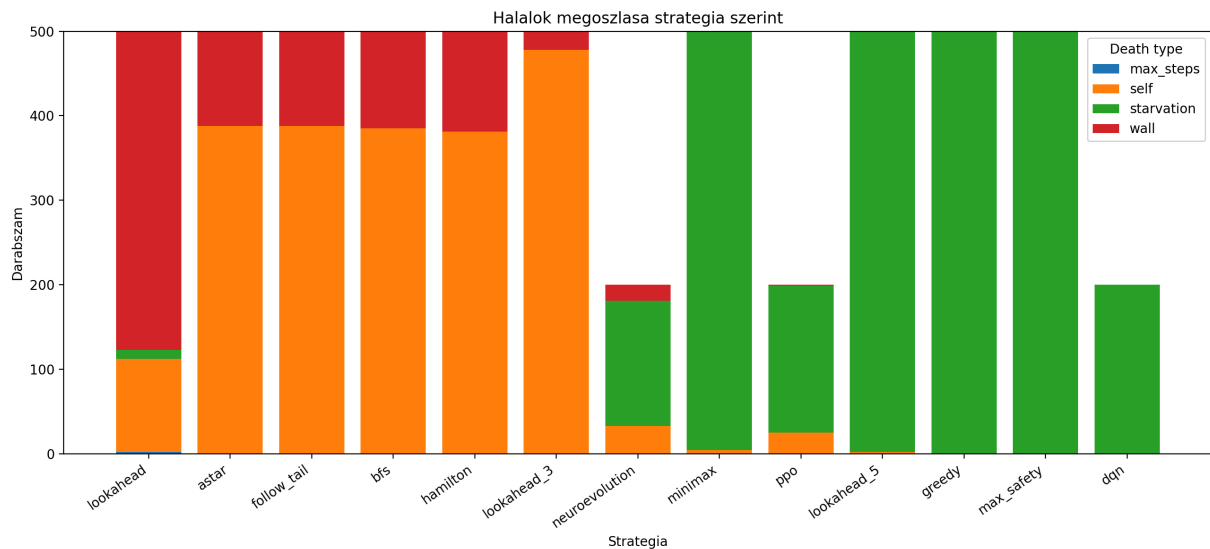
Átlagpont stratégiánként – gyors rangsorolás:



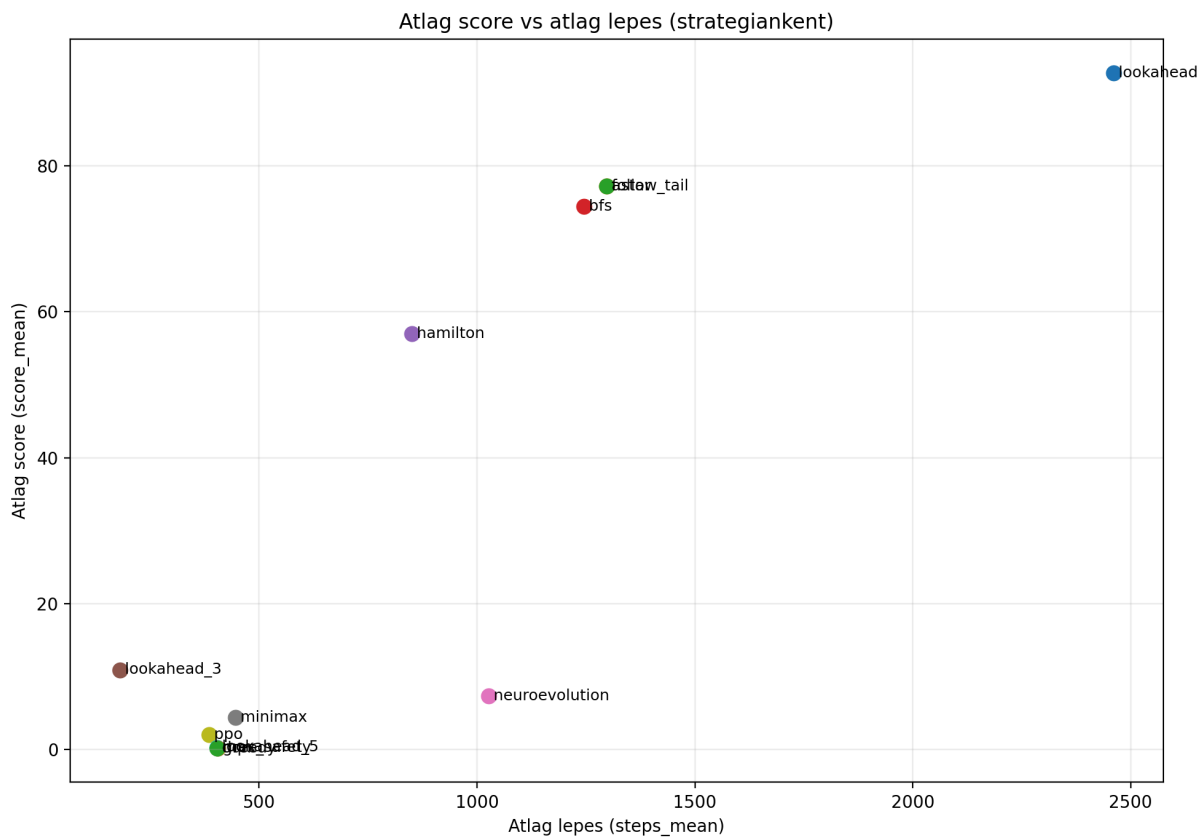
Ponteloszlás – medián, kvartilisok és kiugrók (boxplot):



Halálok megoszlása – wall / self / starvation / max_steps arányok (stacked):



Pont vs. lépésszám – hatékonyság / túlélés trade-off (szórásdiagram):



5.3.8 Záró következtetések (kampány és szakmai útmutató)

Az alábbi pontok a dokumentált **JSON-kampány** és a metrikák **együttes** értelmezése alapján fogalmazódnak — **nem** általános algoritmus-rangsor, hanem **fix pálya**, **fix seed-sorozat**, **fix éhezési cutoff** és **adott runs** mellett érvényes megállapítások.

1. **Heurisztikák és erős teljesítmény:** a [5.3.5](#) szerint a **legerősebb** eredmények explicit szabályokon alapulnak (különösen **lookahead**, **astar**, **follow_tail**, **bfs**,

hamilton). Ezekre jellemző **magas pont** és **100% körüli első étel arány** — vagyis **magas ételvételvezetés, alacsony éhségarány** társulhat hozzájuk. **Greedy, max_safety** és **lookahead_5** ezzel szemben **alacsony pont + domináló éhség** mintát mutatnak, összhangban a **strategyBenchmarkDetail** leírásaival (*óvatosság / nem célvezérelt viselkedés / mélyebb előretekintés hátrányai*).

2. **Tanult stratégiák:** a [5.3.6](#) szerint **NEAT > PPO > DQN**; a **NEAT** a legjobb kompromisszum a három közül, de **nem múlja felül** a top heurisztikákat. A **PPO** és főleg a **DQN** **alacsony 1. étel %-kal** és gyenge átlagponttal illeszkedik az **elakadás, nem elég agresszív célvezetés, rossz jutalom (tanulóknál gyakori)** mintákhoz. **Fontos: rosszabb benchmarkeredmény nem jelenti automatikusan, hogy a módszer rossz** — csak hogy **ebben** a környezetben és **ebben** a konfigurációban a heurisztikus baseline-hoz képest **nem** versenyképes; a javítási irány: jutalom, epizód hossz, megfigyelés finomítása.
3. **Paradigma és hatókör:** **azonos környezetben** a döntési **paradigma** (kézzel írt heurisztika vs. RL vs. neuroevolution) **erősen befolyásolja** a mérhető pontszámot és halálprofil. Állítások **egy pályaméretre, rögzített protokollra** és tárolt **modellverziókra** korlátozódnak — *nem* következtetünk belőlük univerzális „minden Snake példán A jobb, mint B” típusú kijelentésekre.
4. **Vizualizáció:** a **summary** mezők és a **négy generált diagram** (átlag pont, boxplot, halálmegoszlás, pont vs. lépés) együtt segítik felismerni a **nagy szórású** vagy **instabil** stratégiákat (boxplot és szórás: vannak-e kiugróan jó és korán elvérző futások).
5. **Kontroll és reprodukálhatóság:** ajánlott **minden stratégiára azonos runs** melletti **kontroll-kampány** ([5.2.3](#)), hogy a mintanagyság **ne torzítsa** a viszonylagos sorrendet; az összefoglaló **runs** **mezője** ezt közvetlenül dokumentálja. Tanult modelleknel a **reprodukálhatóság** a mentett súlyok és környezet rögzítésétől függ ([3.4.3](#)).

6. A rendszer felhasználása es funkcionalitások

Ez a fejezet a **felhasználói élmény** szempontjából írja le, mit tud a webes alkalmazás: mely **képernyők** váltanak egymásra, milyen **adatok** és **háttér szolgáltatások** (Node backend, AI szolgáltatás) kapcsolódnak hozzájuk. A megvalósítás fájljai: főleg `frontend/src/App.tsx`, `frontend/src/ui/*`, `frontend/src/view/*`, `frontend/src/hooks/*`, `frontend/src/api.ts`, `frontend/src/io/*`.

Tipikus használati sorrend: **főmenü** → (opcionálisan **regisztráció** / **bejelentkezés**) → **beállítások** vagy közvetlen **játék indítás** → **játék** → **eredmények** / **profil**; a **statisztika** külön nézetben érhető el a benchmark adatokhoz.

6.1 Áttekintés: egyoldalas navigáció és screen állapot

Az alkalmazás **egyoldalas (SPA)** felület: a gyökérkomponens egy **screen** React állapot szerint vált a nézetek között. A lehetséges értékek típusa:

```
type Screen = 'menu' | 'settings' | 'results' | 'statistics' | 'game' | 'login' | 'regi
```

- **Menü és környezet:** `menu`, `settings`, `results`, `statistics` – mindegyiknél megjelenik a közös fejléc (Header).
- **Auth:** `login`, `register` – szintén fejléccel; siker után tipikusan `menu`.
- **Profil:** `profile` – csak akkor érdemes megnyitni, ha a felhasználó be van jelentkezve (a `menü` és a fejléc is ezt feltételezi).
- **Játék:** `game` – ekkor a fejléc alatt **HUD**, **Canvas**, **vezérlő gombok** és (INIT fázisban) **Start** réteg látható; nincs külön „játék route”, csak ez az állapot.

Párhuzamosan a játékhoz tartozik a **config** (`GameConfig`, `loadConfig` induláskor) és a **gameMode**: `'player' | 'ai'`. A főmenü játékmód választó gombjai és a beállítások **MI** fájl határozzák meg, melyik módot indítja el a „Játék indítása” / mentés utáni indítás.

6.2 Navigáció és fejléc (Header)

A **Header** minden fenti nézetben látható (kivéve, hogy a játék képernyőn is megjelenik a fejléc a HUD felett). Funkciói:

- **Cím:** „Snake – MI” (alkalmazás azonosítás).
- **Auth:** kijelentkezve **Bejelentkezés** és **Regisztráció** gombok → `login` / `register` képernyő. Bejelentkezve **felhasználónév**, **Profil**, **Kijelentkezés** (`AuthContext.logout()` – törli a `snake_token` és `snake_user` kulcsokat a `localStorage`-ból; a játék-konfiguráció és a téma **nem** törlődik).
- **Téma:** váltógomb a **ThemeContext.toggleTheme**-re kötve (részletek: [6.5 Témaválasztás](#)).

A fejléc **minden „menü jellegű” nézet tetején** rögzített szerepet tölt be: a felhasználó innen látja, be van-e jelentkezve (felhasználónév + profil), és egy kattintással válthat témát anélkül, hogy elhagyná az aktuális képernyőt. A játék nézetben is megjelenik, így a **HUD** alatt marad a közös branding és az auth elérés.

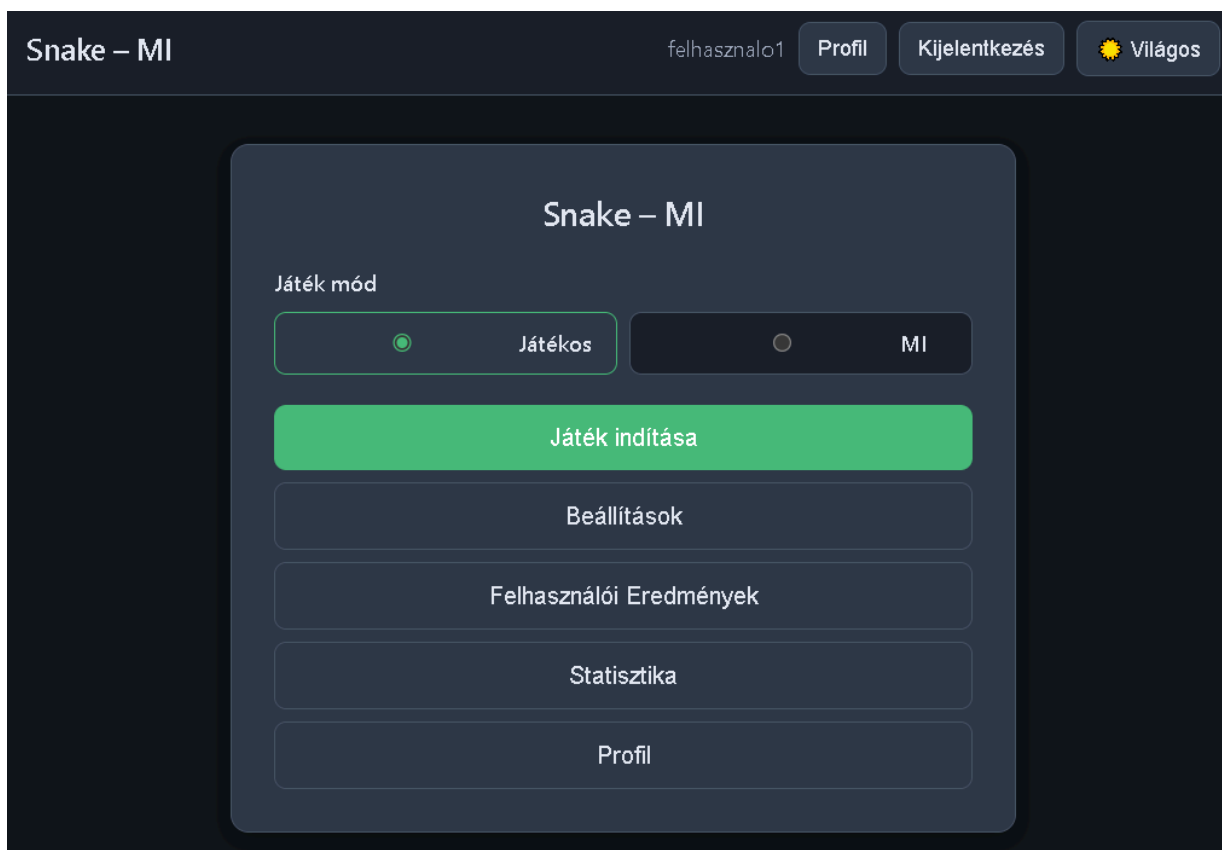


6.1. ábra – Fejléc (Header): alkalmazáscím, vendég módban a Bejelentkezés és Regisztráció, jobbra a világos/sötét téma váltó.

A fejléc callbackjei az App.tsx-ben egy helyen vannak összerakva (onLoginClick, onRegisterClick, onProfileClick), így minden képernyő egységesen ugyanazt az auth navigációt kapja.

6.3 Főmenü (MainMenu)

A **MainMenu** a belépési pont indulás után (`screen === 'menu'`). Vizuálisan **két fő blokk** határozható meg: felül a **játékmód** (játékos vs. MI), alatta a **navigációs gombok** (játék indítása, beállítások, eredmények, statisztika, opcionálisan profil). A felület szövegei magyarul jelennek meg; a kiválasztott játékmód határozza meg, hogy a „Játék indítása” melyik `gameMode` értékkel hívja a háttérlogikát.



6.2. ábra – Főmenü (MainMenu): játékmód gombok a módhoz, gombok a többi nézethez és a játék indításához.

- **Játék mód választó: Játékos vagy MI.** Ez **nem** írja felül önmagában a `config` mezőit; csak azt határozza meg, hogy a „Játék indítása” gomb melyik móddal hívja az `onStartGame(config, mode)`-ot. A pályaméret, tick, seed és MI stratégia a **Beállításokban** állítható (lásd [6.6 Beállítások](#)).
- **Játék indítása:** `startGameWithConfig` logika: `setConfig + saveConfig → createGame` a megadott seeddel (`config.seed` hiányában `Date.now()`), `gameMode`

- beállítása, `screen → 'game'`.
- **Beállítások:** `screen → 'settings'`.
- **Eredmények:** ha van JWT (token), az App meghívja a `fetchScores()` API-t; a válasz `ScoreEntry` sorait `StoredScore` formára mapeli (köztük `ai_strategy → aiStrategy`), `setScores`, majd `results`. Hiba esetén üres lista és mégis megnyílik az eredmények nézet. **Vendég** módban nincs hálózati hívás: `loadScores()` a `localStorage`-ból.
- **Statisztika:** `screen → 'statistics'` (benchmark adatok; részletesen [6.11 Statisztika oldal](#)).
- **Profil:** csak **bejelentkezve** jelenik meg a menüben; `screen → 'profile'`.

6.4 Bejelentkezés, regisztráció és auth perzisztencia

Bejelentkezés (LoginForm): felhasználónév vagy email + jelszó. A kérés a **api.login**-on megy (VITE_API_URL vagy `http://localhost:3000`). Siker esetén **setAuth(token, user)** (AuthContext): `localStorage.setItem('snake_token', ...)`, `localStorage.setItem('snake_user', JSON.stringify(user))`, majd navigáció a menüre.

Regisztráció (RegisterForm): email, felhasználónév, jelszó, megerősítés; validáció és duplikáció a backend válasza szerint. Siker után ugyanúgy token + user tárolás és menü.

A regisztrációs felület **úrlapmezőkből** és egyértelmű **Vissza** / elküldés jellegű akciókból áll; hiba esetén a felhasználó a válaszüzenetből (pl. foglalt név vagy email) tud javítani. A bejelentkezés képernyő **hasonló kompozíciójú**, de csak azonosító (felhasználónév vagy email) és jelszó mezőket kér.

The image shows a registration form titled "Regisztráció" on a dark blue background. It contains four input fields: "E-mail", "Felhasználónév (3–32 karakter)", "Jelszó (min. 6 karakter)", and "Jelszó ismétlése". At the bottom, there are two buttons: a green "Regisztráció" button and a grey "Vissza" button.

6.3. ábra – Regisztráció (*RegisterForm*): email, felhasználónév, jelszó és megerősítés; a Bejelentkezés nézet ugyanebben a stílusban épül fel.

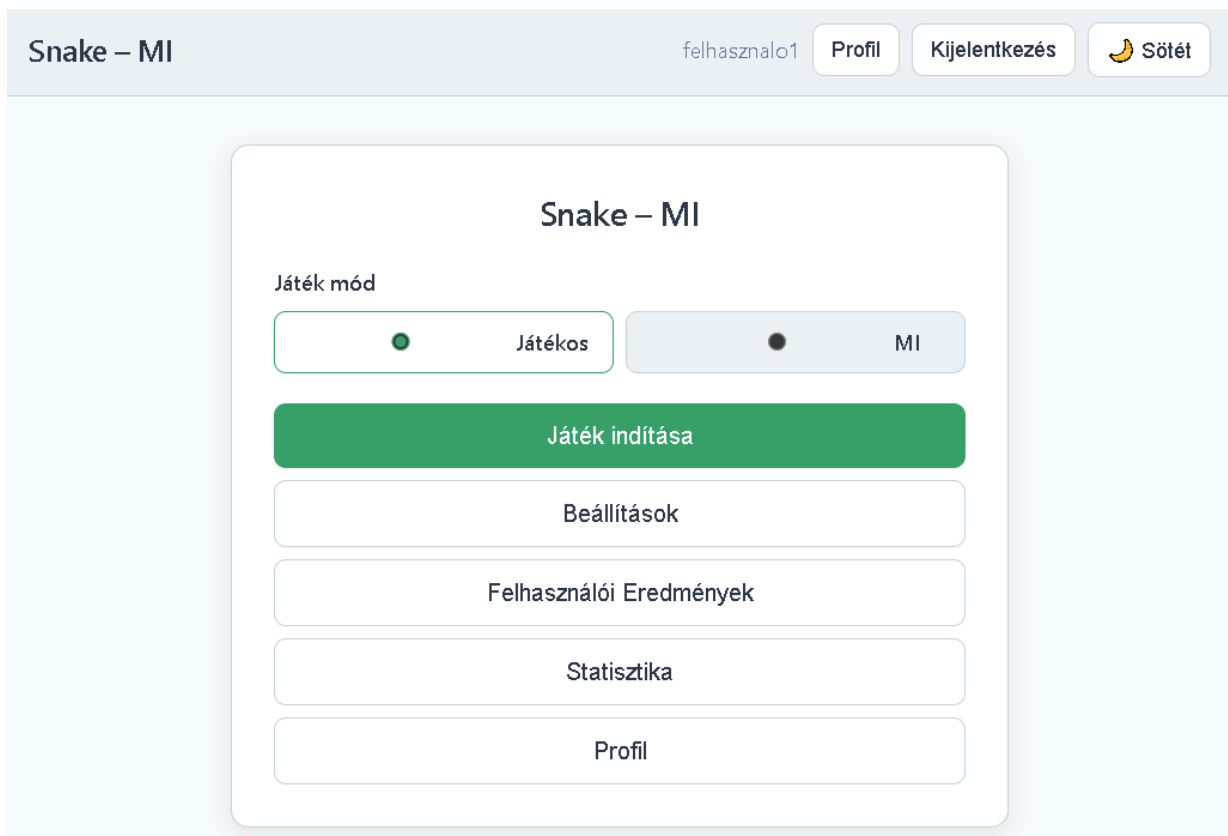
Mindkét űrlapon **Vissza** → menu. Az **AuthProvider** induláskor visszaolvassa a token/user párost; ha van token, de a user hiányzik, **getMe()** próbál szinkronizálni – hiba esetén **logout()**.

6.5 Témaválasztás (világos / sötét)

A **ThemeContext** a `document.documentElement` **data-theme** attribútumát állítja (`light` / `dark`), és a választást **snake_theme** kulccsal menti a `localStorage`-ba. A színek a **theme.css** szelektoraik alapján változnak (`[data-theme="dark"]` / `[data-theme="light"]`).

Fontos megkülönböztetés: a **GameCanvas** 2D kontextusban **közvetlenül** kapja a téma alapján választott színeket (nem CSS változókból olvas), mert a Canvas nem öröklí automatikusan a stíluslap változóit. Így a téma váltás a teljes UI-ra és a játéktér palettájára is hat.

Az alábbi kép a **világos téma** megjelenését mutatja (háttér, szövegek, gombok kontrasztja); a sötét téma ugyanezen elemek **sötétebb háttérrel és invertáltabb szövegszínnel** jelenik meg, a `data-theme` váltás szerint.



6.4. ábra – Világos téma példa a böngészőben: a fejléc és a fő tartalom világos háttérrel jelenik meg.

6.6 Beállítások (Settings, PlayerSettings, AISettings)

A beállítások képernyő **két fület** használ (`role="tablist" / tabpanel`):

1. **Játékos (PlayerSettings)**: **sor- és oszlopszám** (ésszerű tartomány, tipikusan 10–40), **tick időköz** ms-ban (játéksebesség), opcionális **seed** (üres = véletlen indulás mentéskor/indításkor). **Mentés** → `onSave (saveConfig + setConfig)`. **Játék indítása** → játékos módban azonnali indítás ugyanezzel a konfigurációval.
2. **MI (AISettings)**: ugyanazok a pálya / tick / seed mezők, plusz **stratégia** legördülő: az **AI_STRATEGIES** lista (`strategies.ts`) minden eleméhez név és rövid leírás; a kiválasztott azonosító a **`config.ai.strategy`** mezőbe kerül, és **localStorage**-ba mentve marad.

A képernyő **fülstruktúrája** (Játékos / MI) megkönnyíti, hogy a felhasználó ne keverje össze az emberi irányításhoz és a MI-hez tartozó beállításokat; a **Mentés** mindkét fülön a közös `GameConfig` perzisztenciára épül.

Beállítások

Játékos

MI

Sorok

20

Oszlopok

20

Tick (ms)

50

Seed (üres = véletlen)

pl. 42

MI stratégia

Hamilton (spirál) ▼

Egy előre megrajzolt „kör” mentén halad a pályán (spirál: külső perem befelé). Az étel felé csak akkor tér le, ha biztonságos – így garantáltan nem ütközik a falba.

Játék indítása

6.5. ábra – Beállítások (Settings): a Játékos fülön sor/oszlop, tick, seed és mentés; az MI fülön ugyanezek mellett stratégia választás (a képen nem látható másik fül).

Alul **Vissza a főmenübe** zárja a nézetet. Az induláskor betöltött konfiguráció **loadConfig** (io/config.ts).

6.7 Játék képernyő – játékos mód

HUD (`HUD.tsx`): élő **pont**, **kígyó hossz**, **lépésszám**, **tick/s** ($1000 / \text{tickMs}$), **státusz** (INIT → „Készül”, RUNNING → „Fut”, PAUSED → „Szünet”, GAME_OVER → „Vége”). MI módban kiegészül egy **MI:** sorral (lásd [6.8 Játék képernyő – MI mód](#)).

Canvas: a pálya, kígyó és étel rajzolása (`GameCanvas`).

Billentyűzet (globális `keydown` az `App.tsx`-ben, csak `screen === 'game'` esetén):

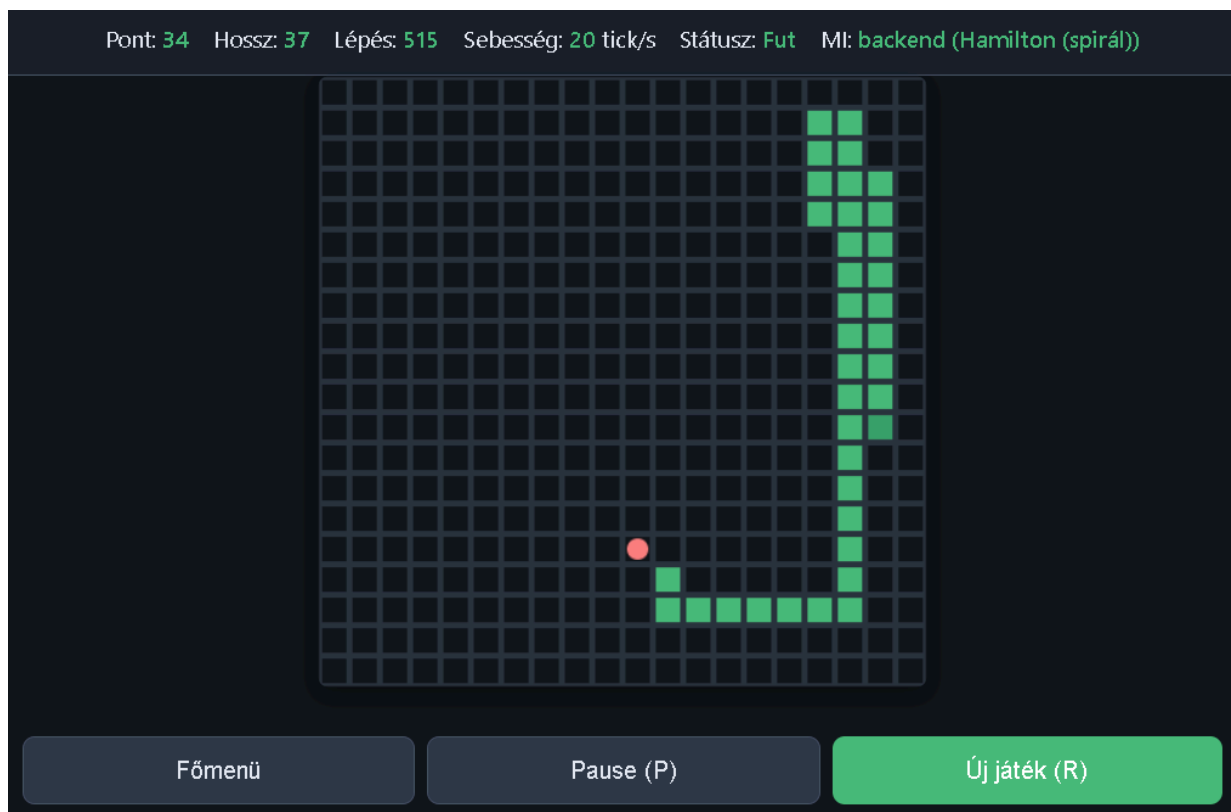
- **Nyilak és WASD:** irány (`setDirection`) – **játékos** módban, ha a fázis **RUNNING** vagy **INIT**.
- **P:** szünet / folytatás (`pauseGame / resumeGame`), ha **RUNNING** vagy **PAUSED**.
- **R:** új játék **ugyanabban** a módban (`startNewGame(gameMode)`).
- **Enter:** INIT fázisban **startGame**.

Egér: INIT alatt egy **Start** gomb is megjelenik átfedésként (`game-start-overlay`); ez ugyanazt csinálja, mint az **Enter**.

Gombok a játék alatt: **Főmenü**, futás közben **Pause (P)**, szünetben **Folytatás (P)**, **Új játék (R)**. **GAME_OVER** után szöveg: „Játék vége. Pont: ...”.

Játékciklus: `useGameLoop` csak akkor aktív, ha `screen === 'game'` és `gameMode === 'player'` – fix intervallumos `tick` hívások.

A következő ábra a **játék nézet** tipikus elrendezését mutatja: **HUD** a fejléc és a vászon között, alatta a **pálya** és a **vezérlő gombok** (főmenü, szünet, új játék). MI módban a HUD „**MI:**” sora és a kapcsolat állapota egészíti ki ugyanezt a képet (lásd [6.8](#)).



6.6. ábra – Játék nézet: pont, hossz, lépésszám, tick/s és státusz; a vászon alatt akciógombok.

6.8 Játék képernyő – MI mód

Ugyanaz a HUD és Canvas, de a léptetést **useAIGameLoop** végzi, ha `screen === 'game'` és `gameMode === 'ai'`. Minden tick után a kliens **WebSocket**en elküldi a pillanatképet (`VITE_AI_WS_URL` vagy `ws://localhost:8000/ws`), stratégia azonosítóval; a válasz **irány** alapján történik a mozgás. Ha nincs kapcsolat, **helyi fallback** (`placeholderStrategy` a `Strategy.ts`-ben).

A HUD ekkor a következőt jeleníti meg (a `getStrategyName` a felhasználóbarát nevet adja):

```
{aiConnected !== undefined && (  
  <span title={aiConnected ? 'ai_service WebSocket' : 'Helyi stratégia'}>  
    MI: <strong>{aiConnected ? `backend (${getStrategyName(aiStrategy)})` : 'helyi'}</span>  
)}
```

6.9 Játék vége és pontszám mentés

Amikor a játékalapot **GAME_OVER** és a képernyő még **game**:

1. **saveScore** (`io/storage.ts`): pont, tick, hossz, ISO dátum, mód (`player / ai`), MI esetén **aiStrategy**.
2. **setScores(loadScores())** – a lokális lista frissítése.
3. Ha van **JWT**, **submitScoreApi** (`POST /api/scores`) – a szerver oldali eredménytáblába; hiba esetén csendes `.catch(() => {})`.

Fejlesztői megjegyzés: az `App.tsx` egy `useEffect`-ben induláskor meghívja **clearScores()**, ami törli a vendég **localStorage** **eredménylistáját**. Éles vagy demonstrációs használat előtt érdemes eldönteni, ezt megtartjuk-e; különben az oldal újratöltése után a lokális eredmények nem maradnak meg.

```
useEffect(() => {  
  clearScores()  
}, [])
```

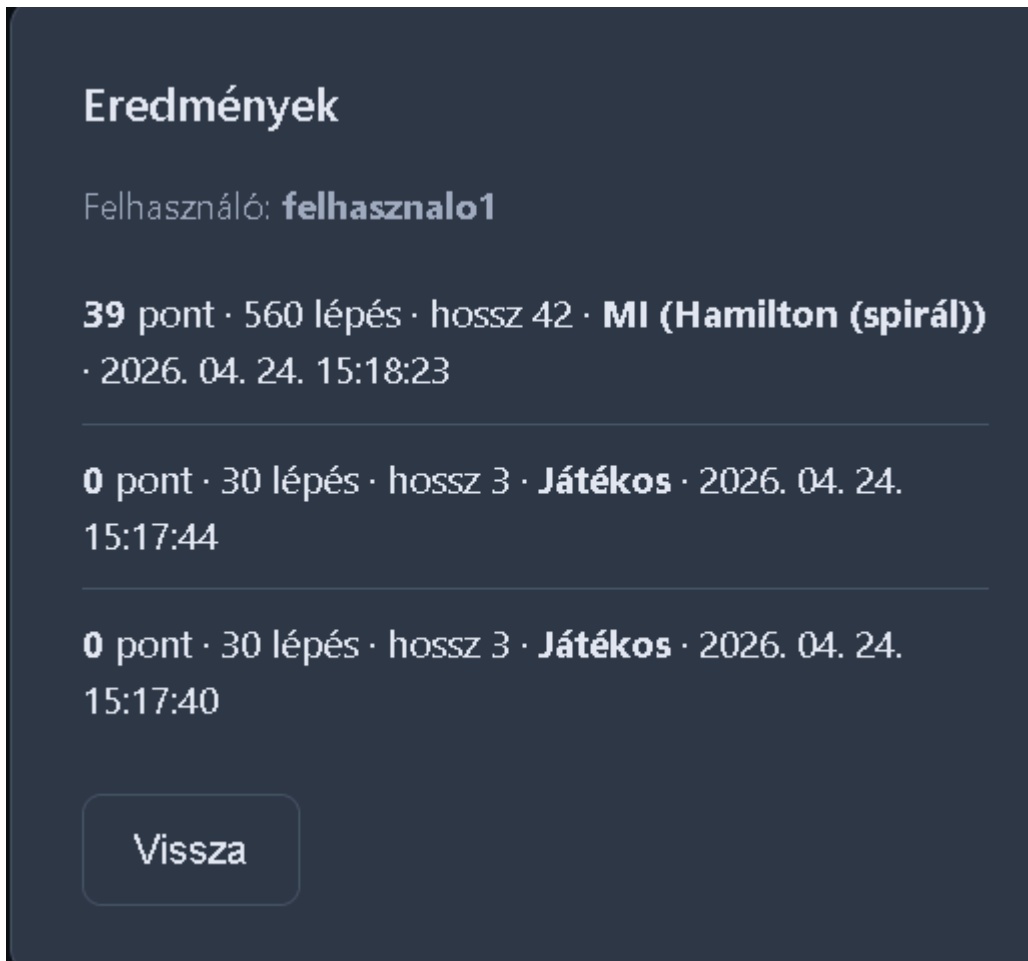
6.10 Eredmények oldal (Results)

Az **Results** a főmenü **Eredmények** gombjáról nyílik; propként kapja a már betöltött **scores** tömböt (az App intézi a backend vs. `localStorage` forrást, lásd [6.3 Főmenü](#)).

- Bejelentkezett felhasználónál a lista felett „**Felhasználó: ...**” megjelenik.
- Minden sor: **pont, lépés, hossz, Játékos** vagy **MI** (stratégia megjelenített neve), **helyi idő** (`toLocaleString('hu-HU')`).
- Legfeljebb **20** sor (`scores.slice(0, 20)`).
- Üres lista: „**Még nincs mentett eredmény.**”
- Alul, mint a **Vissza** a főmenübe gomb található.

Az eredménylista **táblázatos** formában összefoglalja a lokálisan vagy szerverről betöltött futásokat; a sorok **csökkenő idő vagy pont** szerint értelmezhetők a megjelenítés implementációja szerint (max.

20 elem). Bejelentkezett felhasználónál a fejléc alatti „Felhasználó: ...” szöveg egyértelművé teszi, hogy a szerver oldali eredmények jelennek meg.



6.7. ábra – Eredmények (Results): pont, lépés, hossz, mód (Játékos / MI stratégia) és időbélyeg oszlopokkal.

6.11 Statisztika oldal (Statistics) – összefoglaló

A **Statistics** komponens (`frontend/src/ui/Statistics.tsx`) a főmenü **Statisztika** gombjáról nyílik (`screen === 'statistics'`). Célja, hogy a Python oldalon futtatott **stratégiai benchmark** összefoglalót **böngészőben**, táblázatban és kiegészítő grafikonokkal lehessen átnézni — a játékos mód **játékonkénti** eredménylistájától ([6.10 Eredmények](#)) független nézet. A számok forrása **nem** a Node backend, hanem az **AI szolgáltatás**: a kliens a `api.ts` `fetchBenchmarkSummaries()` függvényével hívja a `GET /benchmark/summaries` végpontot (bázis URL: `VITE_AI_HTTP_URL`, fejlesztőben tipikusan `http://localhost:8000`). A válasz `summaries: BenchmarkSummary[]` típusú objektumok (stratégia, runs, pálya, `max_steps`, `seed_base`, pont- és lépésszámok, `death_counts`, `reached_first_food`), opcionálisan `benchmark_dir` (a szerver által beolvasott eredménymappa elérési útja) és hiba esetén `error` mező. A megvalósítás részletei és egy `useEffect` részlet: [3.2.6 Statisztika oldal](#); a JSON szerkezet és a szerver oldali beolvasás: [5.2.3 Kimenet: JSON és összefoglaló](#); a metrikák szakmai értelmezése: [5.3 Eredmények részletes értelmezése](#). A generált ábrák szerepe a dokumentumban: [5.3.7 Vizualizáció: generált grafikonok](#) és az [ábrák jegyzéke](#); a kapcsolódó táblázatok: [táblázatok jegyzéke](#).

Szöveges szekciók (felhasználói útmutató)

Az oldal tetején **statikus, magyar nyelvű** magyarázó blokkok követik egymást — ezek nem a szervertől jönnek, hanem a komponens JSX-ébe vannak építve:

1. **Mit mutat ez az oldal?** — szimulált játékok összegzése, **20×20** pálya és **előre megadott magok** (a benchmark `seed_base + index` szerinti értelmezése); hangsúly: **nem** „ember elleni verseny”, hanem **összehasonlítható** döntési logikák és **bukási okok** (fal, önmaga, éhség, lépéslimit).
2. **Metodika (hogyan értelmezd a számokat)** — felsorolás: **runs**, átlag vs. medián pont, átlag lépésszám, halál okok (külön kiemelve, hogy az **éhség** a futtató által **leállított**, hosszú ideje stagnáló pont miatti esemény), **0%** halál oszlopok jelentése (hiányzó JSON-kulcs), **első étel %** és az „elakadás”.
3. **Heurisztikák vs. tanult stratégiák** — rövid narratíva és a „**rosszabb benchmark nem jelenti automatikusan, hogy a módszer rossz**” figyelmeztetés, baseline (heurisztika) szerepe.
4. **Rövid értelmezés (tipikus minták)** — négy minta: magas pont + alacsony éhség; alacsony pont + éhség; sok fal/önütközés; nagy szórás (boxplot/scatter).

Betöltési hibák és üres állapot

- **Hálózati / HTTP hiba** (fetch kivétel vagy nem OK válasz): piros **stats-error** szöveg, javaslat: indítsa az AI szolgáltatást (<http://localhost:8000>) és töltsse fel a **benchmarks/results** mappát.
- **Sikeres válasz, de üres summaries**: üzenet, hogy nincs **strategy_benchmark_*.json** a szerver által visszaadott **benchmark_dir** alatt — ez gyakran akkor fordul elő, ha a mérés még nem futott le a gépen, vagy az elérési út nem egyezik a projekt **benchmarks/results** mappájával.

Összefoglaló táblázat

Ha van legalább egy összefoglaló sor:

- **Rendezés**: minden **oszlopfejléc** gomb; alapértelmezés szerint **score_mean csökkenő** (erősebb stratégiák felül). Új oszlopra váltáskor a stratégia oszlop **növekvő**, a többi numerikus oszlop **csökkenő** irányban indul; ugyanarra az oszlopra kattintva **vált** növekvő ↔ csökkenő. A rendezés **magyar locale** szerinti stratégianév-hasonlítást is használ a **Stratégia** oszlopnál.
- **Oszlopok**: Stratégia (megjelenített név + monospace **strategy** azonosító), Futás, Átlag pont, Medián, Átlag lépés, 1. étel %, Fal %, Önmaga %, Éhség %, Lépéslimit %. A halálarányok **death_counts[k] / runs * 100**, hiányzó kulcs esetén **0%** (deathPct).
- **Szűrés**: <details> blokk, „**Stratégiák szűrése (kijelölt / összes)**” felirattal. Gyors gombok: **Mind**, **Egyik sem**, **Csak heurisztikák**, **Csak neurális hálók** — utóbbi a dqn, ppo, neuroevolution azonosítókra szűkít (NEURAL_STRATEGY_IDS). Egyébként **stratégiáinként jelölőnégyzet**; ha minden kijelölés törlődik, „**Nincs megjeleníthető stratégia**” üzenet.
- **Sorinterakció**: sorra **kattintás** (vagy billentyűzetten **Enter / Szóköz**): kinyit / bezár egy **részletes panelt** ugyanahhoz a stratégiához. A sor **aria-expanded, tabIndex={0}**, **role="button"** értékekkel támogatja a kényelmesebb használatot. Ha a kijelölt stratégia **kiszűródik**, a panel automatikusan bezáródik.

Részletes panel (stratégiánként)

Megnyitáskor (ha van `strategyBenchmarkDetail` bejegyzés):

- **Röviden:** a `getStrategyById / strategies.ts` szerinti rövid leírás.
- **Működési logika és Miért lehetnek ilyenek a számok?** — a `strategyBenchmarkDetail.ts` szövege (általános minták; a konkrét számok a JSON-tól függenek).
- **A te gépeden mért összefoglaló:** runs, pálya rows×cols, max_steps, seed_base, átlag/medián pont, átlag lépés, első étel %, halál okok darabszámban.
- **Részletek bezárása** gomb.

Ismeretlen stratégia kulcshoz a modul **általános** tartalék szöveget ad.

Grafikonok (PNG)

A négy fix fájlnev: `score_mean_bar.png`, `score_distribution_boxplot.png`, `death_distribution_stacked.png`, `score_vs_steps_scatter.png`. Forrás: `benchmarkPlotUrl` → `GET /benchmark/plots/{filename}`. A képek **gombokba** vannak ágyazva: kattintás **lightbox**-ot nyit (`role="dialog"`, **Escape** bezár, `document.body.overflow` átmenetileg rejtve). Bezárás: háttérre kattintás, × gomb vagy **Escape** — mindez a felhasználói útmutató szövegében is szerepel az oldalon.

Navigáció

Alul a **Vissza** gomb az `onBack` callbackre kötve tér vissza a hívó nézethez (tipikusan a főmenü).

6.12 Profil oldal (Profile)

Csak **bejelentkezett** felhasználónak érdemes megnyitni (a menü így is kínálja). Funkciók:

- **Felhasználónév módosítása** – `updateUsername`, majd `loadUser()` a friss adatokért.
- **Jelszó módosítás** – jelenlegi / új / megerősítés, kliens oldali ellenőrzések; `updatePassword`.
- **Saját eredmények** – `fetchScores()` ugyanazzal a megjelenítési logikával, mint az eredményeknél.

A profil oldal **több szekcióra** tagolja a műveleteket: fent az **azonosító módosítás** (felhasználónév), középen a **jelszócsere** (jelenlegi + új jelszó mezők), alul a **saját eredmények** listája – így egy helyen kezelhető a fiók és a játéktörténet.

Profil

fel@fel.fel · felhasználó1

Felhasználónév módosítása

felhasználó1

Mentés

Jelenlegi jelszó

Új jelszó

Új jelszó ismétlése

Jelszó mentése

6.8. ábra – Profil (*Profile*): név- és jelszó módosítás, valamint a felhasználó eredményeinek összefoglalója.

Alul, mint eddig a **Vissza** a főmenübe gomb található.

7. Következtetések

Ez a fejezet **összefoglalja**, mit érdemes levonni a dokumentált rendszerből és mérésekből, **milyen hatókörben** érvényesek az állítások, és hogyan illeszkednek a szakmai gyakorlathoz. Összhangban áll a [5.3.8 Záró következtetések](#) alfejezettel.

7.1 Megvalósítások

A dolgozat **egy játszható webes alkalmazást** és **mögötte egy kutatható, reprodukálható kiértékelési keretet** valósít meg. A kód **monorepóban** él: **React + TypeScript** frontend (játék, auth, téma, Statisztika nézet), **Node.js + Express + SQLite** backend (JWT, pontszámok), **Python FastAPI** AI szolgáltatás (HTTP + WebSocket döntés, benchmark fájlok kiszolgálása). A játék **magja** (frontend/src/core) UI-tól független tiszta függvényekre épül; a Python oldal és a benchmark ugyanarra az **állapot** → **irány** szemléletre illeszkedik (**Strategy** interfész a kliensen; **Strategy.next_move** a szolgáltatásban). **Perzisztencia**: opcionális fix **seed**, játék-konfiguráció és téma **localStorage**-ban; eredményeknél **mód** és opcionális **ai_strategy**.

A dokumentált **benchmark kampány** és a **webes Statisztika** együttesen azt mutatja, hogy a **fix protokollban** (részletek: [5. Üzembe helyezés és kísérleti eredmények](#), különösen [5.2 Kísérleti eredmények, mérések](#) és [5.3 Eredmények részletes értelmezése](#)) a **heurisztikák** – különösen az erős **előretekintéses** és útvonalkereső baseline-ok – **stabil, magas teljesítményt** adnak. A **tanuló** megközelítések közül a **NEAT** a dokumentált beállítások mellett **ésszerű kompromisszum**; a **PPO** és főleg a **DQN** ebben a konfigurációban **elmarad** a top heurisztikáktól, összhangban a halálprofil- és **első étel** mutatók szöveges értelmezésével ([5.3 Eredmények részletes értelmezése](#)). Ez **nem** „általános algoritmus-rangsor”: a következtetések **egy pályaméretre, rögzített seed_base, max_steps, éhezési cutoff** és adott **runs** mellett érvényesek. **Nem** következtethetünk belőlük arra, hogy „algoritmus A minden Snake változaton jobb, mint B” – kerülendő a **túlzott általánosítás**.

A mérések **összevethetőségét** veszélyezteti, ha a heurisztikák és a tanult stratégiák **különböző futásszámmal** szerepelnek: érdemes **közös kontrollfutást** tervezni (**azonos runs** minden stratégiára). **Átlag** (score_mean) **önmagában** félrevezető rangsor lehet; a **medián** és az **eloszlás szélessége** (boxplot a generate_benchmark_plots.py kimenetén) ugyanilyen fontos ([6.11 Statisztika oldal](#)). **Halálprofil**: sok **starvation** nem egyenlő automatikusan „jó túléléssel”; gyakran **gyenge pontkonverzió** vagy **leállítási küszöb** jele. A **self / wall** arányt a **pontszámmal együtt** kell olvasni.

7.2 Hasonló rendszerekkel való összehasonlítás

A **szakirodalomban** és más Snake / RL rendszerekben a **jutalom**, a **megfigyelés**, a **pálya**, a **tanítási büdzsé** és a **lépés-limit** gyakran más, ezért **közvetlen számszerű összevetés** a saját eredményekkel **nem** mindig értelmes. A dokumentált **DQN / PPO / NEAT** eredmények **erősen függenek** a fenti tényezőktől ([3.4.6 Tanuló környezet és algoritmusok](#)); a mért számok **nem** a módszerek elméleti maximumát jelentik, hanem a **jelenlegi, rögzített beállítások** melletti teljesítményt.

Inferencia és tanítás szétválasztása: ha hiányzik a **modellfájl** vagy a függőség, a szolgáltatás **Greedy fallbackre** vált ([3.4.3 Tanult stratégiák](#)); ekkor a benchmark a stratégia neve alatt **más viselkedést** mérhet, mint tiszta tanult policy esetén. Reprodukálhatósághoz **fájlverziók** és környezet rögzítése szükséges.

Óvatos megfogalmazás: kerülendő az „a tanuló algoritmus **alkalmatlan**” típusú kijelentés. Helyette: „**ebben a környezetben és erőforrás-keretben gyengébb, mint a dokumentált heurisztikák**” – összhangban a **webes Statisztika** szövegével és azzal, hogy **rosszabb benchmark** önmagában **nem** mondja ki, hogy a módszer más beállítással ne működhetne jobban.

7.3 Továbbifejlesztési lehetőségek

A rendszer jelenlegi állapota **összefoglaló demonstráció és összehasonlító keret** szerepét tölti be: a továbblépés nem egyetlen „hibajavítás”, hanem **priorizált fejlesztési sáv** – üzemeltetés, **adat- és kísérlet-fegyelem**, tanulási módszertan, felhasználói réteg és **mérnöki minőség** mentén. Az alábbi alcímek a dolgozatban már bemutatott elemekre ([3. architektúra](#), [4. tervezés](#), [5. mérések](#), [6. felhasználás](#)) építenek, és **konkrét, megvalósítható** irányokat fogalmaznak meg realisztikus hatókörrel.

7.3.1 Rendszer, telepítés és üzemeltetés

A teljes élményhez a **három szolgáltatás** (frontend, Node backend, AI – tipikusan **:8000**) párhuzamosan fut; a **MI mód** és a **benchmark-alapú statisztikai nézet** különösen az AI réteg **elérhetőségétől és válaszidejétől** függ ([6.8](#), [6.11](#), [4.1.5](#)). Érdemes **egységes indítási / leállítási** szkriptet vagy konténer-alapú összerakást (Docker Compose) fontolóra venni, hogy a **függőségek és portok** ne maradjanak „fejben tartott” részletek. **Éles** vagy **osztott** környezetben megjelenhetnek további kérdések: **fordított proxy, HTTPS, CORS**, időtúllépések és a **WebSocket** életciklusa ([3.4.4](#)) – ezek dokumentálása és automatizált ellenőrzése csökkenti a **helyfüggő hibák** számát.

JWT és auth esetén a **JWT_SECRET** és a token lejárat politikája **nem maradhatnak fejlesztői mintaértékeken** ([3.3.2](#)); éles telepítésnél a **kulcskezelés**, naplózás és **sebezhetőségi frissítések** (függőségek) önálló feladatcsomag. Összefoglalva: a következő lépés gyakran nem új algoritmus, hanem **ismételhető deploy és üzemeltethetőség**.

7.3.2 Benchmark-adatok, szinkron és reprodukálhatóság

A **benchmarks/results/** könyvtárban lévő **JSON-kampányok** és a belőlük készült **PNG plotok** a statisztikai nézet **valós adatforrásai** ([5.2](#), [3.5](#)). Más gépen vagy új klón után, **szinkron nélkül**, a felület **üres** vagy **elavult** adatot mutathat. Javasolt irány: **verziózott futtatási jegyzék** (commit hash, Python / CUDA környezet, futtatott parancs, seed-stratégia), és opcionálisan **összecsomagolt artefakt** (eredmény + konfiguráció + generált ábrák) a reprodukálhatóságért.

Összehasonlításbeli pontosság: kevés futásnál a **véletlenszerűség** nagy szórást ad; statisztikai megoldásként **több ismétlés**, **bootstrap konfidencia-intervallum** vagy **rangsorolás IQR / medián** szerint részletesebb képet ad, mint az önmagában vett átlag ([5.3](#), [7.2](#)). Hosszabb távon **egységes runs séma** és **exportálható jelentés** (pl. CSV + diagramok) megkönnyíti a **szakdolgozaton túli** folytatást is.

7.3.3 Tanulási környezet, jutalomstruktúra és hiperparaméterek

A tanuló ág teljesítménye **nem** csak az algoritmus-választástól függ: a **megfigyelés**, a **jutalom**, a **lépés-limit**, az **étel** elhelyezése és a **starvation** (hosszú kísérletezés büntetése) együtt alakítja a policy-t ([3.4.6](#), [4.2](#)). Konkrét továbbfejlesztési irányok többek között:

- **Pálya- és ütemvariancia:** más **rácsméret**, fal / wrap szabályok, vagy **több pályasablon** változatosabb általánosítást kérhet – ugyanakkor nő a **tanítási költség** és a kiértékelés

komplexitása.

- **Tanítási költség explicit mérése:** ugyanazon eredmény eléréséhez szükséges **wall-clock idő**, **interakciós lépcső** vagy **epizód** számának dokumentálása lehetővé teszi a **hatékonyság vs. végső pontszám** trade-off írásos összevetését ([5.3.4](#)).
- **Hiperparaméter-keresés:** tanulási ráta, hálóarchitektúra, **exploráció–exploitáció** egyensúly (pl. entropia együttható PPO-nál) – rendszerezett **rács- vagy szűk keresési kampány**, nem egyszeri „kézi próbálkozás”.
- **Megfigyelés és jutalom továbbfinomítása:** ha a policy **beragad** lokális mintákba (pl. körözés), a jutalom **ritka jelek** (pl. távolság az ételtől, ütközés előtti állapot) finomhangolásával vagy **curriculum** (kisebb pályán indulás) javítható lehet.

Fontos szemlélet: **rosszabb benchmark** önmagában **nem zárja ki**, hogy más beállítással vagy hosszabb tanítással a tanuló módszer utolérje a heurisztikákat ([7.2](#)); a továbbfejlesztés tehát **mérhető hipotézisekhez** kötődik, nem ígérethez.

7.3.4 Frontend, auth és felhasználói perzisztencia

A felület **kényelmes demonstrációt** ad, ugyanakkor **bizonyos részletei fejlesztői döntést** igényelnek éles vagy hosszabb távú használatra. A frontend indulásakor végrehajtott **clearScores()** **törölheti** a vendég **localStorage** eredménylistáját ([6.9](#)) – előadás, demo vagy valós használat előtt érdemes **szándékosan** rögzíteni: vendég **vs.** regisztrált felhasználó, **szerveroldali** eredménytárolás ([6.10](#), [3.3](#)) és **ütközésmentes** szinkron. További irány lehet **offline-barát** viselkedés (cache stratégia), **akadálymentesség** (kontraszt, billentyű navigáció már részben a témával érintett – ld. [6.5](#)) és **hibajelzések** egységesítése, ha az AI szolgáltatás átmenetileg nem elérhető.

7.3.5 Minőségbiztosítás, tesztelés és folyamatok

A rendszer moduljai (React állapot, Express útvonalak, Python stratégia-regisztráció) **bővíthetők**: új belépési pont a **STRATEGIES** táblához, új metrika a benchmark-lánchoz ([3.4.5](#), [3.5](#)). A gyors bővítés ára gyakran a **regresszió** – ezért **egységtesztek** (különösen **determinisztikus** játékszabályokra és segédfüggvényekre), **integrációs** próbák a REST / WebSocket szerződésre, és **CI** futtatás már **következő lépcsőként** realiztikusak. Kiegészítő szempontok: **típusosság** szigorítása (TypeScript / hint-ek), **naplózás** egy szinten a Python oldalon is, és **függőség-felülvizsgálat** biztonsági szempontból.

8. Irodalomjegyzék

1. **Sutton, R. S. – Barto, A. G.:** *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press, 2018.
2. **Bellman, R.:** *A Markovian Decision Process*. Journal of Mathematics and Mechanics 6(5), 679–684 (1957).
3. **Watkins, C. J. C. H. – Dayan, P.:** *Q-learning*. Machine Learning 8(3–4), 279–292 (1992).
4. **Mnih, V. et al.:** *Human-level control through deep reinforcement learning*. Nature 518, 529–533 (2015).
5. **van Hasselt, H. – Guez, A. – Silver, D.:** *Deep Reinforcement Learning with Double Q-learning*. Proceedings of the AAAI Conference on Artificial Intelligence 30(1), 2094–2100 (2016).
6. **Schulman, J. et al.:** *Proximal Policy Optimization Algorithms*. arXiv:1707.06347 (2017).
7. **Raffin, A. et al.:** *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Journal of Machine Learning Research 22(268), 1–8 (2021).
8. **Stanley, K. O. – Miikkulainen, R.:** *Evolving Neural Networks through Augmenting Topologies*. Evolutionary Computation 10(2), 99–127 (2002).
9. **Dijkstra, E. W.:** *A note on two problems in connexion with graphs*. Numerische Mathematik 1(1), 269–271 (1959).
10. **Hart, P. E. – Nilsson, N. J. – Raphael, B.:** *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics 4(2), 100–107 (1968).
11. **Knuth, D. E. – Moore, R. W.:** *An Analysis of Alpha-Beta Pruning*. Artificial Intelligence 6(4), 293–326 (1975).
12. **Paszke, A. et al.:** *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 32, 8024–8035 (2019).
13. **Kingma, D. P. – Ba, J.:** *Adam: A Method for Stochastic Optimization*. Proceedings of the International Conference on Learning Representations (ICLR), 2015.
14. **Goodfellow, I. – Bengio, Y. – Courville, A.:** *Deep Learning*. MIT Press, 2016.
15. **Russell, S. – Norvig, P.:** *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2020.

9. Függelék

Ez a fejezet a dolgozathoz kapcsolódó kiegészítő anyagokat foglalja össze.

9.1 Forráskód és dokumentáció (GitHub repó)

A projekt **forráskódja** és **dokumentációja** elérhető a következő **GitHub** repositoryban: https://github.com/GyorgyAkos/snake_projekt.

A repó többek között az alábbiakat tartalmazza:

- a teljes projekt forráskódja (frontend/, backend/, ai_service/, benchmarks/, docs/);
- futtatáshoz szükséges függőséglisták (package.json, requirements.txt);
- dokumentációs állományok (docs/*.md, beleértve a specifikációt és a szakdolgozati leírásokat);
- benchmark kimenetek és ábrák (benchmarks/results/, benchmarks/results/plots/).

9.2 Kiegészítő anyagok

A dolgozat törzsszövegéből terjedelmi okok miatt kimaradó, de a reprodukálhatóságot és továbbfejlesztést támogató anyagok:

- modellfájlok és tréning-kimenetek (ha elérhetők) az ai_service/models/ könyvtárban;
- benchmark JSON és PNG kimenetek a benchmarks/results/ mappában.

9.3 Reprodukálhatósági megjegyzés

A függelék célja, hogy a projekt később is önállóan futtatható és ellenőrizhető legyen. A forráskód beszerzése tipikusan a GitHub tároló **git clone** parancsával történik; a pontos függőségek és indítási lépések a repó dokumentációjában (pl. README, docs/) találhatók. A rendszer újraépítéséhez szükséges további információk (struktúra, benchmark fájlok helye) a törzsben és ebben a fejezetben együtt megtalálhatók.